Lecture 02 — Replacement Selection, Huffman Trees, and k-Way Merging

1. Lecture Map

Lecture 01 introduced external merge sort and tournament trees. This lecture uses those ideas to improve both major parts of an external sort:

> Tip
> **Lecture flow:** generate longer initial runs with a loser tree → merge unequal runs in an optimal order → connect optimal merge trees to prefix codes and Huffman coding → organize buffers for an efficient k-way merge.

The three main performance goals are:

> Important
> - Overlap disk input, disk output, and CPU processing.
> - Reduce the number of initial runs by increasing their average length.
> - Reduce total merge work by choosing a good merge tree and buffer layout.

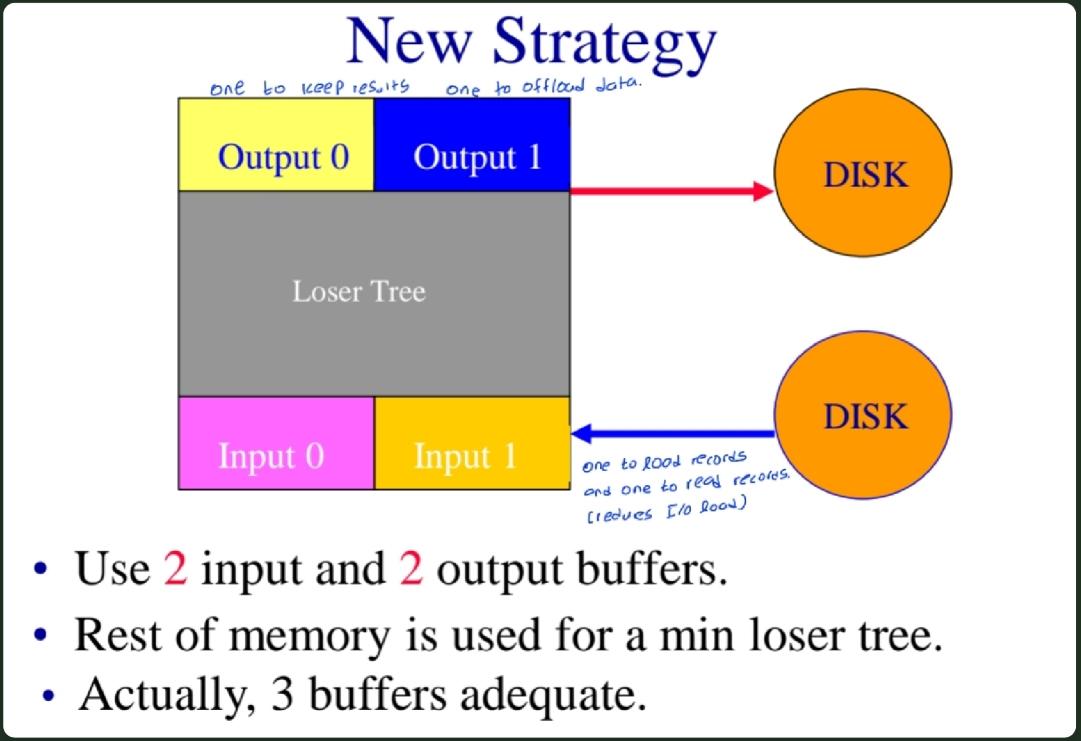
2. Generating Longer Runs with a Loser Tree

The basic run-generation method from Lecture 01 loads k records into memory, sorts them, and writes a run of length k. This produces approximately n/k runs.
A better method, commonly called **replacement selection**, keeps a min loser tree in memory and replaces each record as soon as it

is written. A new record may join the current run if it preserves sorted order; otherwise, it is saved for the next run.

Memory organization

- Two input buffers allow one buffer to be processed while another is filled from disk.
- Two output buffers allow one buffer to be filled while another is written to disk.
- The remaining memory stores a min loser tree.
- The lecture notes that three buffers can be sufficient, although the four-buffer layout makes the overlap easy to visualize.

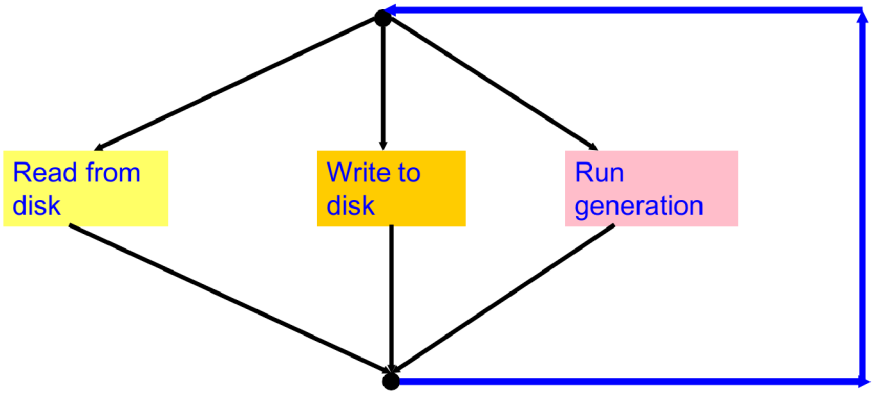


The loser tree occupies most of memory, while input and output buffers connect it to disk.

In steady-state operation, the following activities proceed concurrently:

1. Read the next input block from disk.
2. Generate the current sorted run using the CPU.
3. Write a completed output block to disk.

Steady State Operation



- **Replacement-selection algorithm**

Assume the loser tree has k external nodes.

1. Fill the tree with k input records and initialize it as a min loser tree.
2. Remove the minimum active record and append it to the current run.
3. Read the next input record and compare it with the record most recently written:
   - If `newRecord >= lastOutput`, insert it as an **active** record for the current run.
   - If `newRecord < lastOutput`, mark it as **frozen** because writing it now would break the sorted order. It will participate in the next run.
4. Replay the loser tree to find the next minimum active record.
5. When every record in the tree is frozen, close the current run, unfreeze the records, and begin a new run.
6. Continue until the input is exhausted and all records have been written.

Conceptually, every frozen record is treated as larger than every active record, regardless of its actual key. This ensures that

active records finish the current run before any frozen record is selected.

```text
lastOutput = -∞
initialize min loser tree with k records

while records remain in the tree:
    if every player is frozen:
        close current run
        unfreeze all players
        lastOutput = -∞

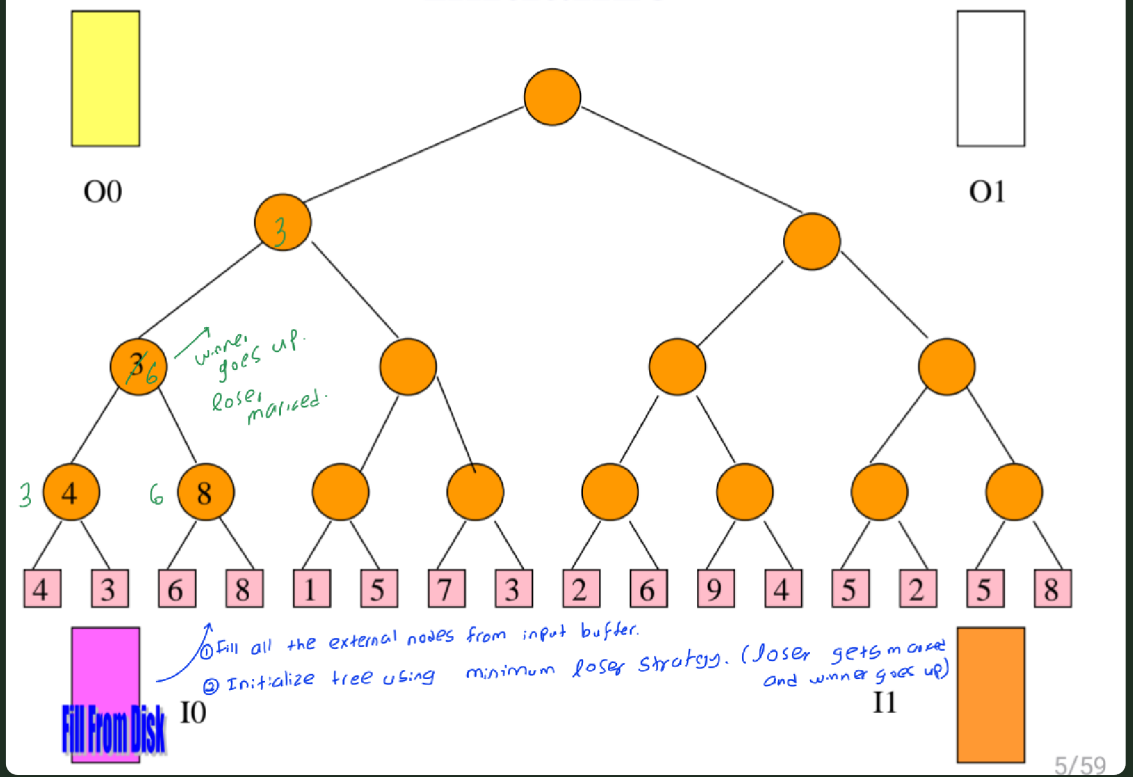
    winner = minimum active record
    write winner to current run
    lastOutput = winner.key

    if another input record exists:
        replacement = read next record
        if replacement.key < lastOutput:
            freeze replacement for the next run
        replace winner with replacement
    else:
        remove the winner's player

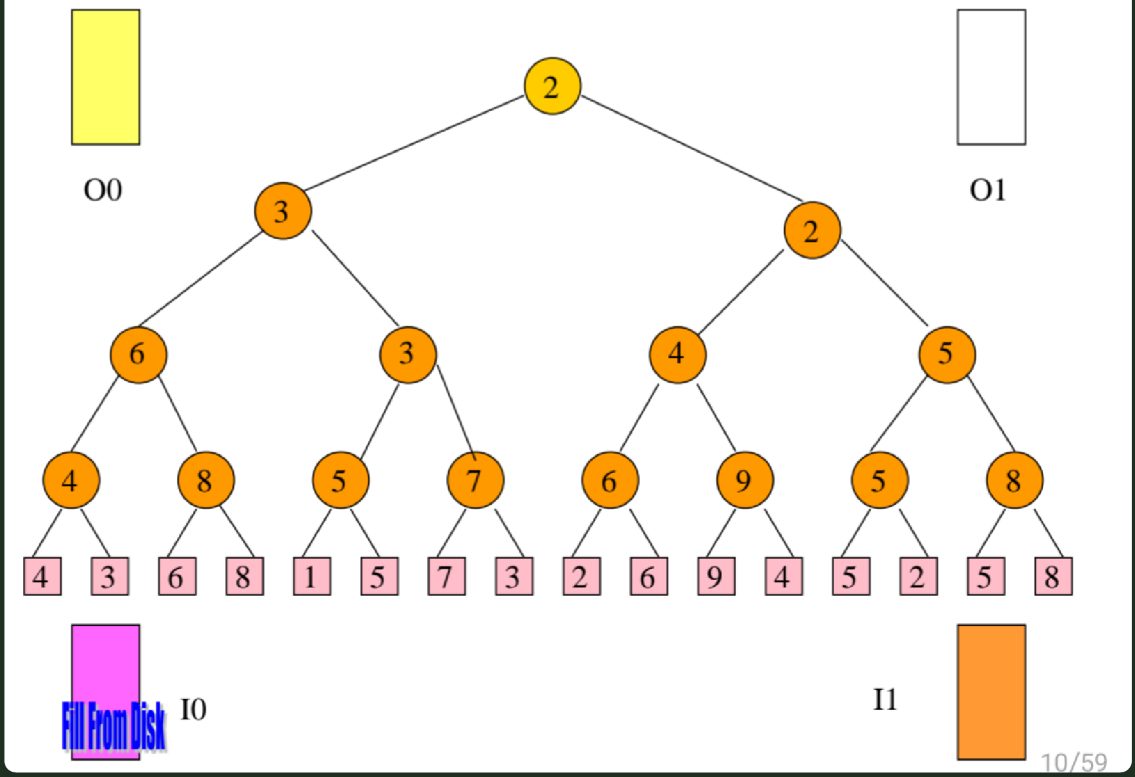
    replay the loser tree
```

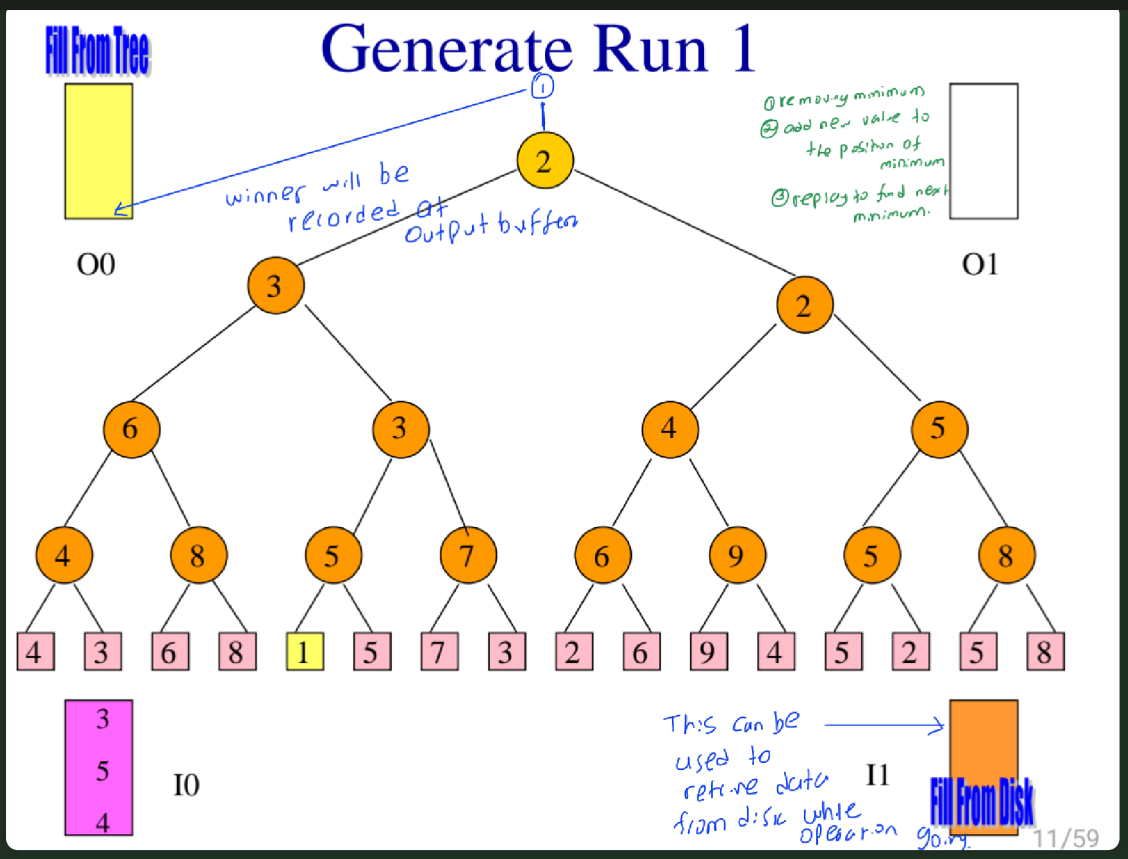
Small example

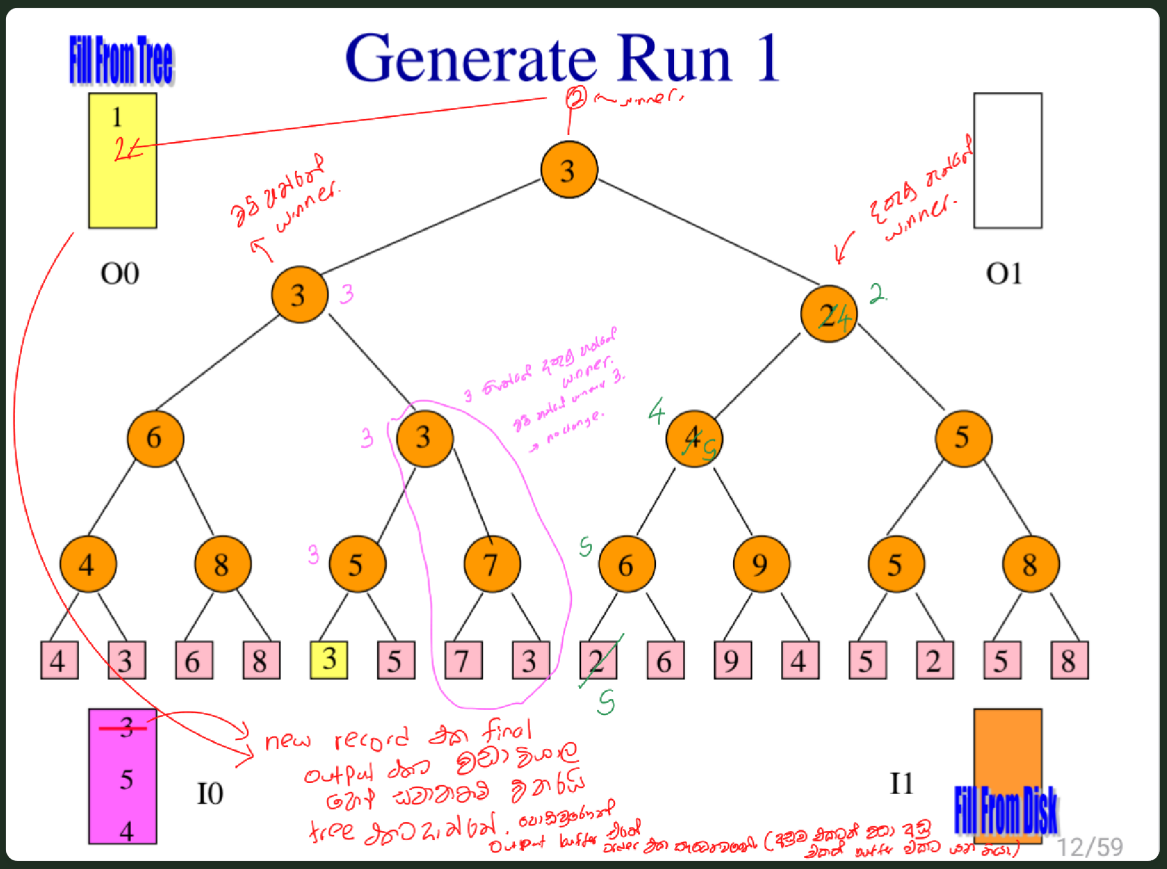
Initialize

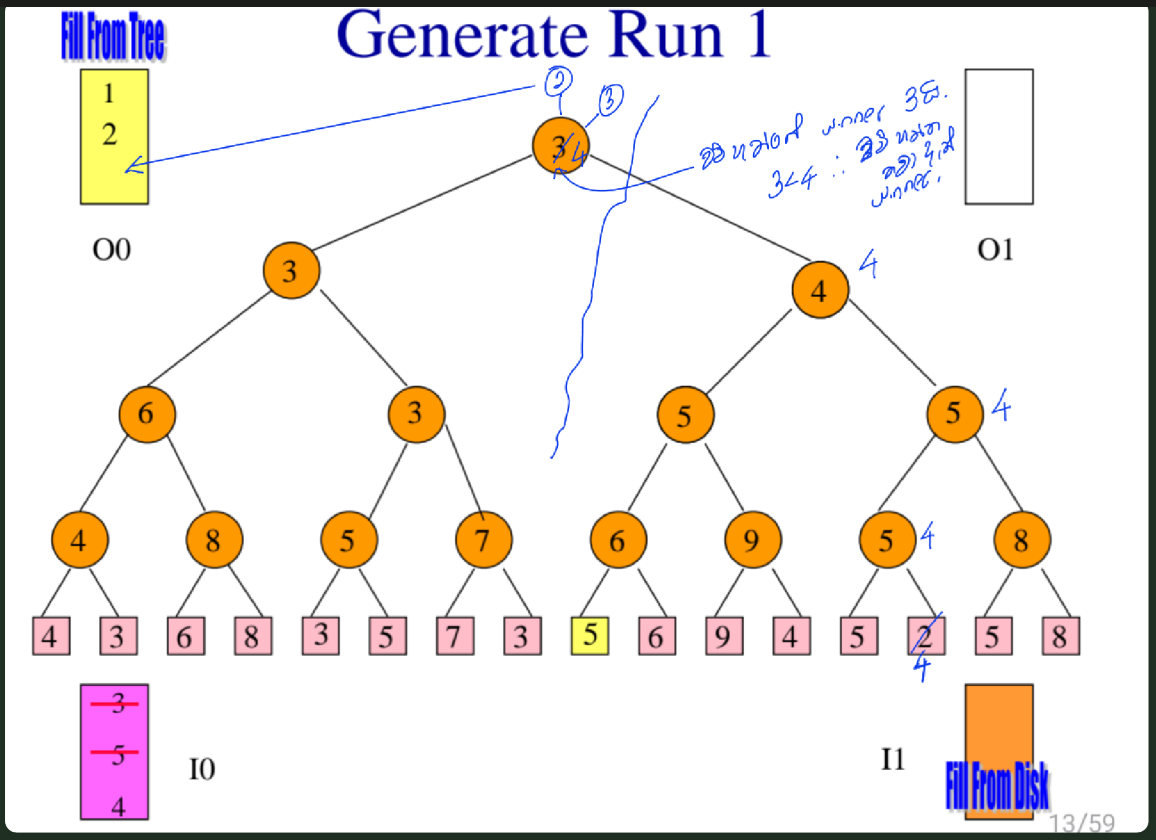


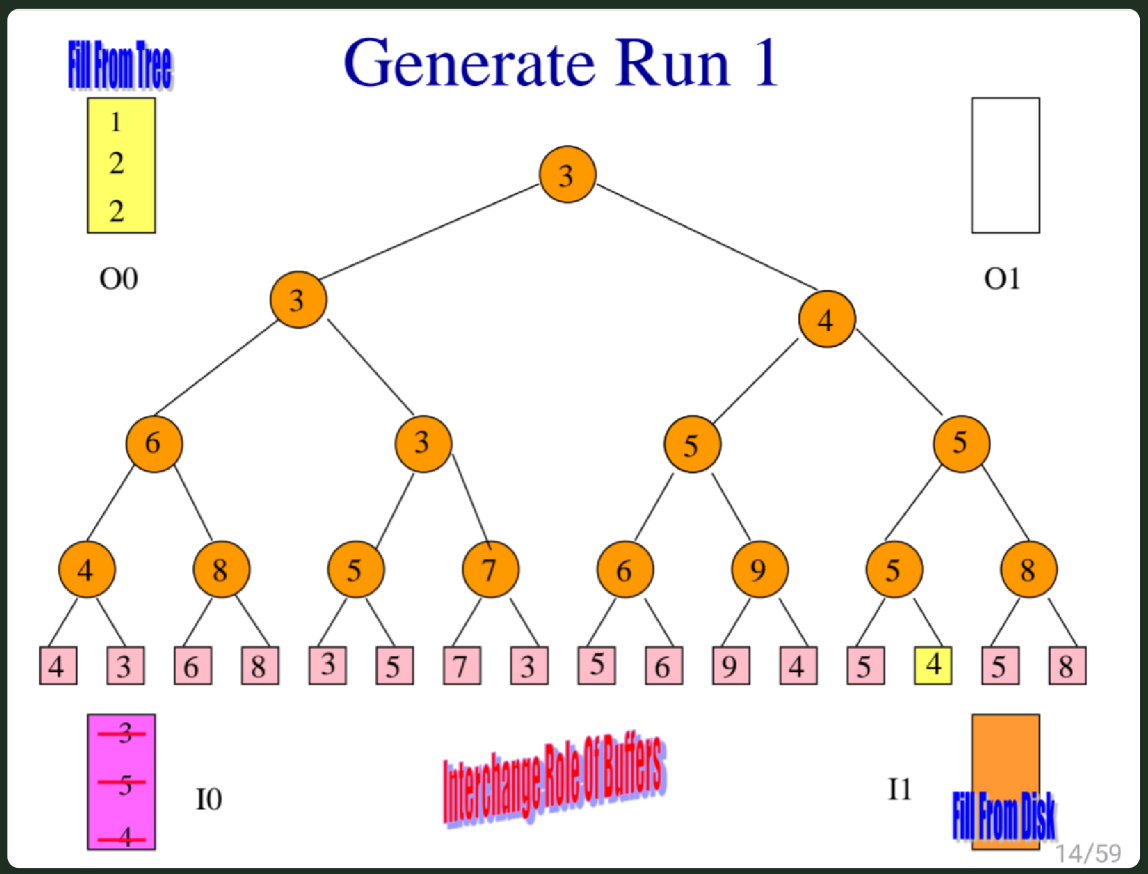
Initialize

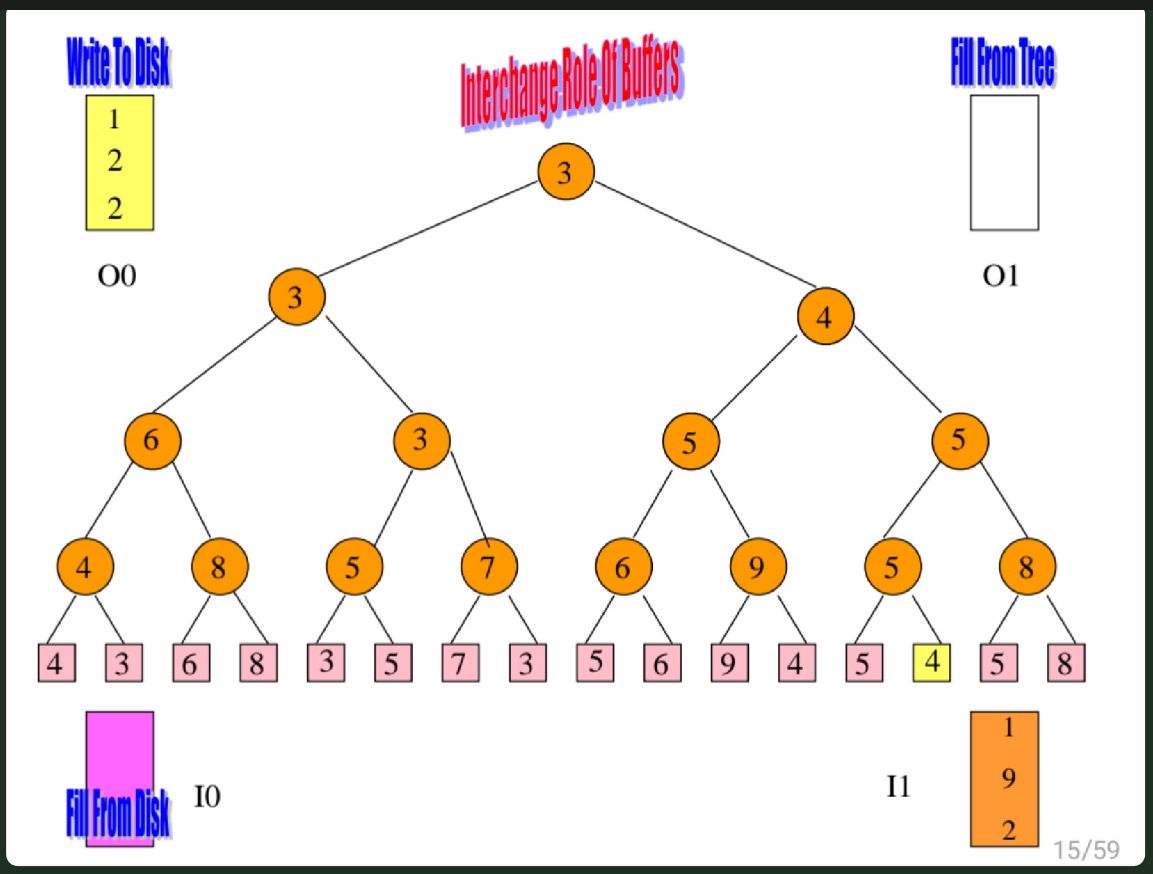


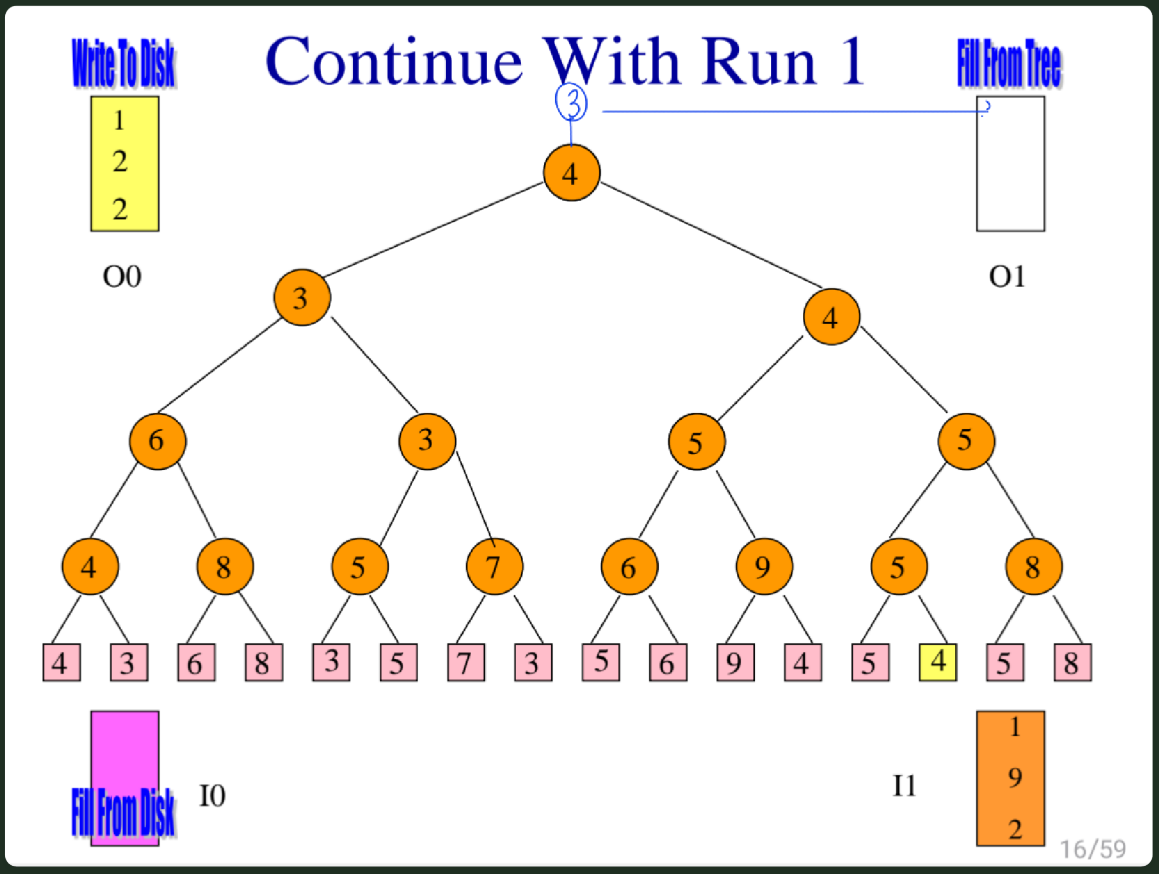

Fill From Tree
Generate Run 1
winner will be recorded at output buffer
O0
O1
I0
I1
This can be used to retrive data from disk while operation going.
Fill From Disk
11/59

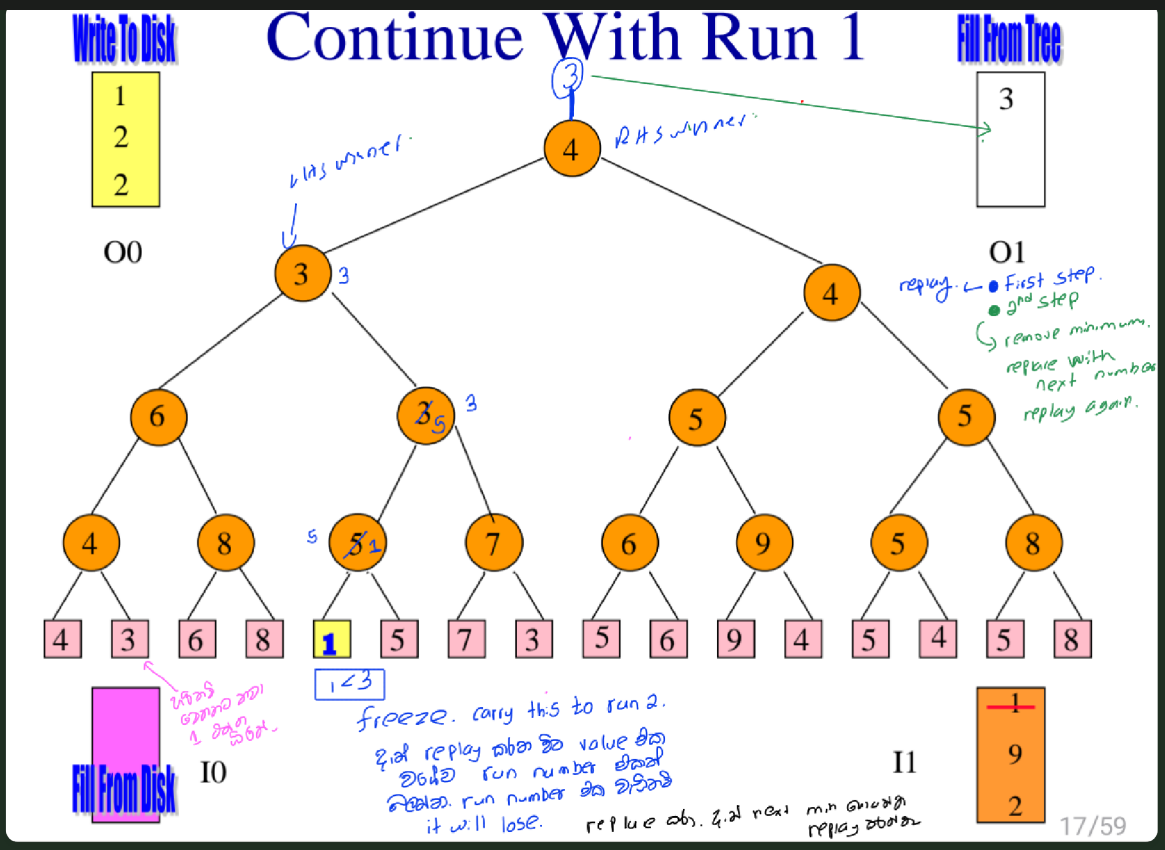

Fill From Tree
Generate Run 1
O0
O1
I0
I1
Fill From Disk
12/59

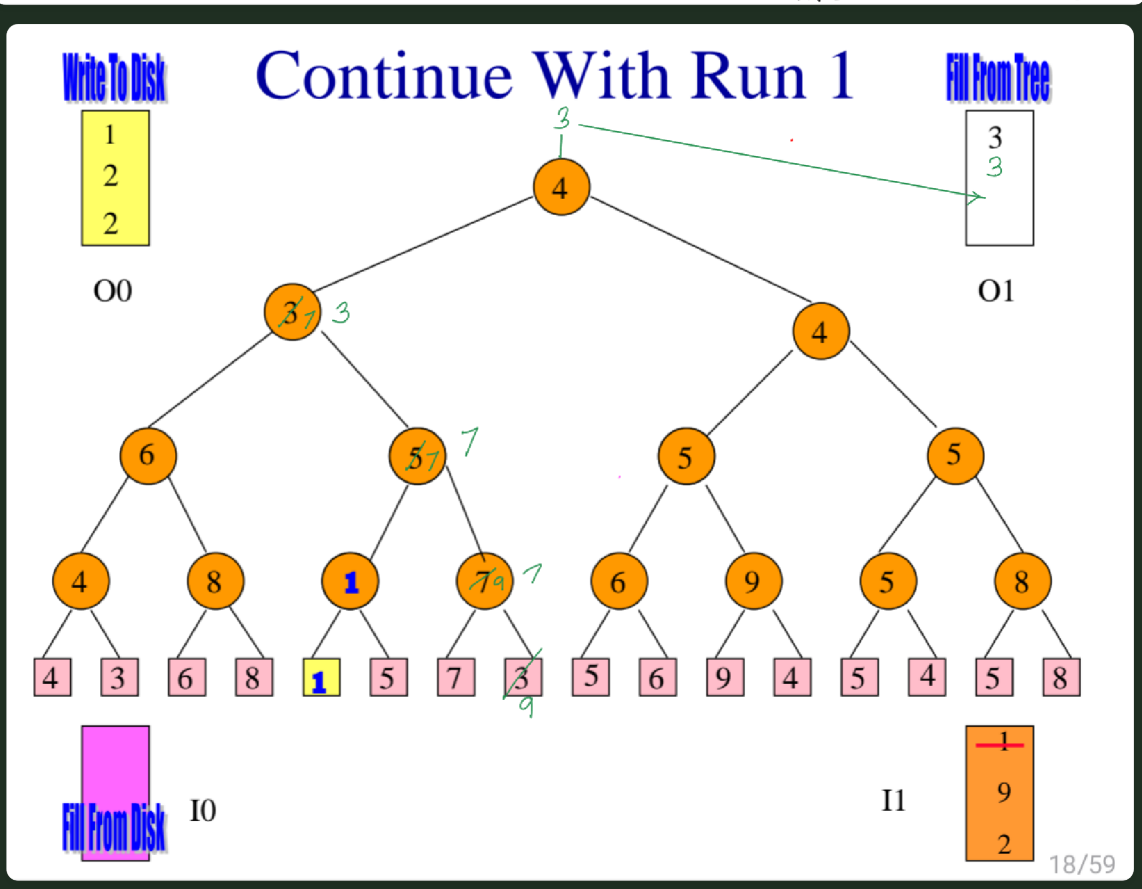

Fill From Tree
Generate Run 1
1
2
O0
O1
3
3
4
6
3
5
5
4
8
5
7
6
9
5
8
4
3
6
8
3
5
7
3
5
6
9
4
5
2
5
8
3
5
4
I0
I1
Fill From Disk
13/59

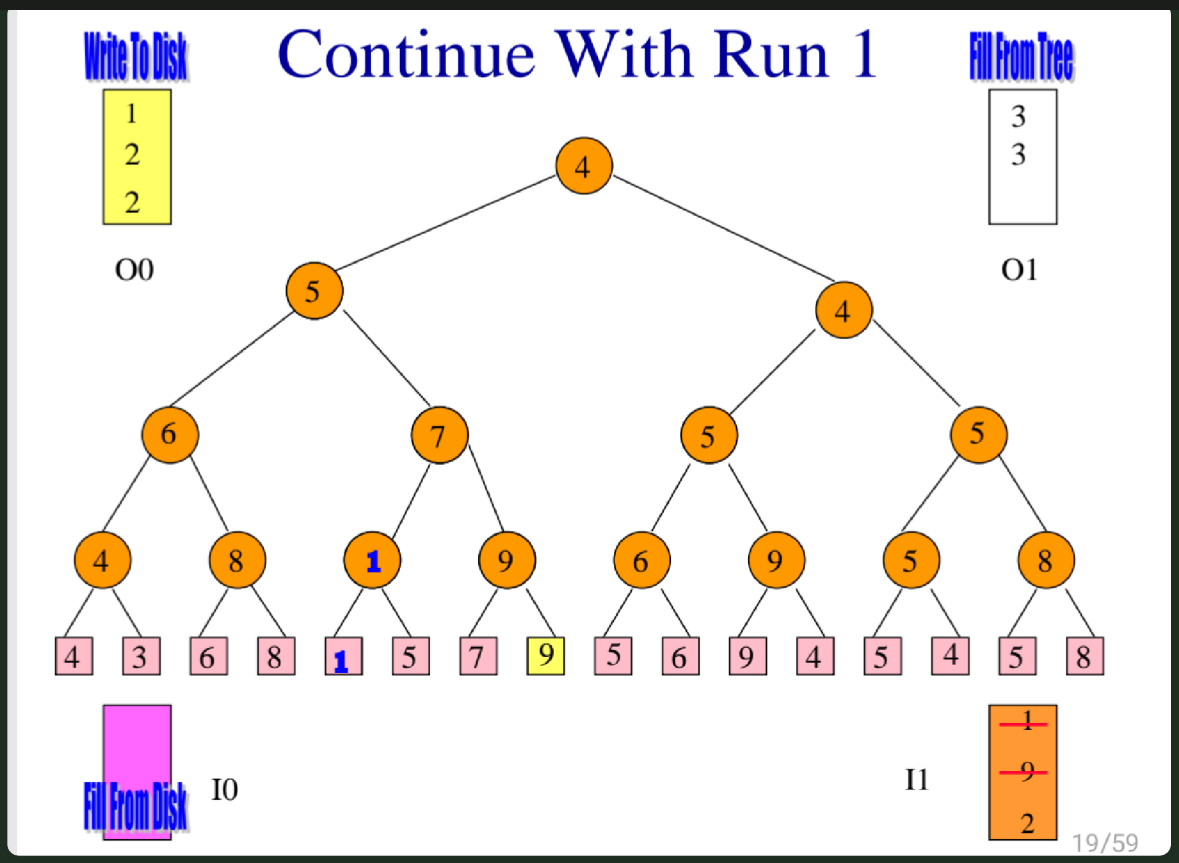

Fill From Tree
Generate Run 1
1
2
2
O0
O1
3
3
4
6
3
5
5
4
8
5
7
6
9
5
8
4
3
6
8
3
5
7
3
5
6
9
4
5
4
5
8
3
5
4
I0
Interchange Role Of Buffers
I1
Fill From Disk
14/59

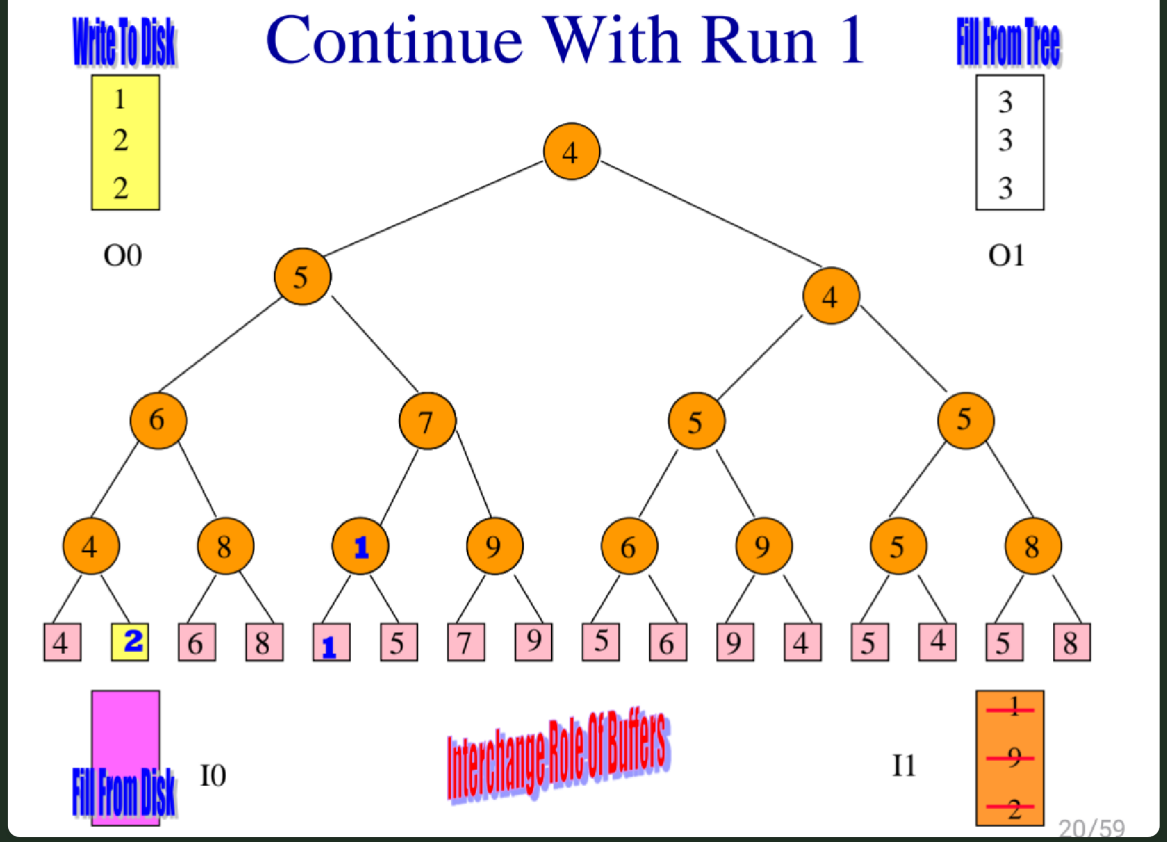

Write To Disk
1
2
2
O0
Interchange Role Of Buffers
Fill From Tree
O1
3
3
4
6
3
5
5
4
8
5
7
6
9
5
8
4
3
6
8
3
5
7
3
5
6
9
4
5
4
5
8
Fill From Disk
I0
I1
1
9
2
15/59

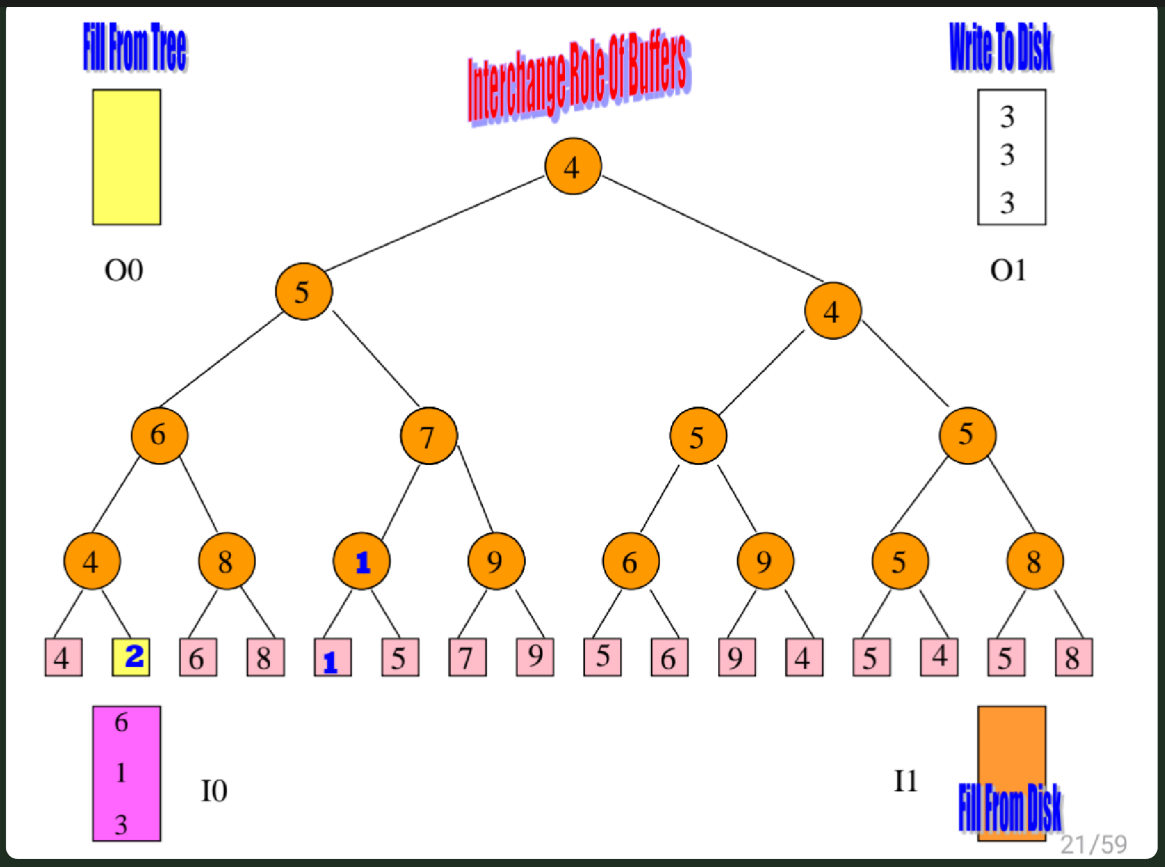

Write To Disk
Continue With Run 1
Fill From Tree
1
2
2
3
4
O0
O1
3
4
6
3
5
5
4
8
5
7
6
9
5
8
4
3
6
8
3
5
7
3
5
6
9
4
5
4
5
8
Fill From Disk
I0
I1
1
9
2
16/59

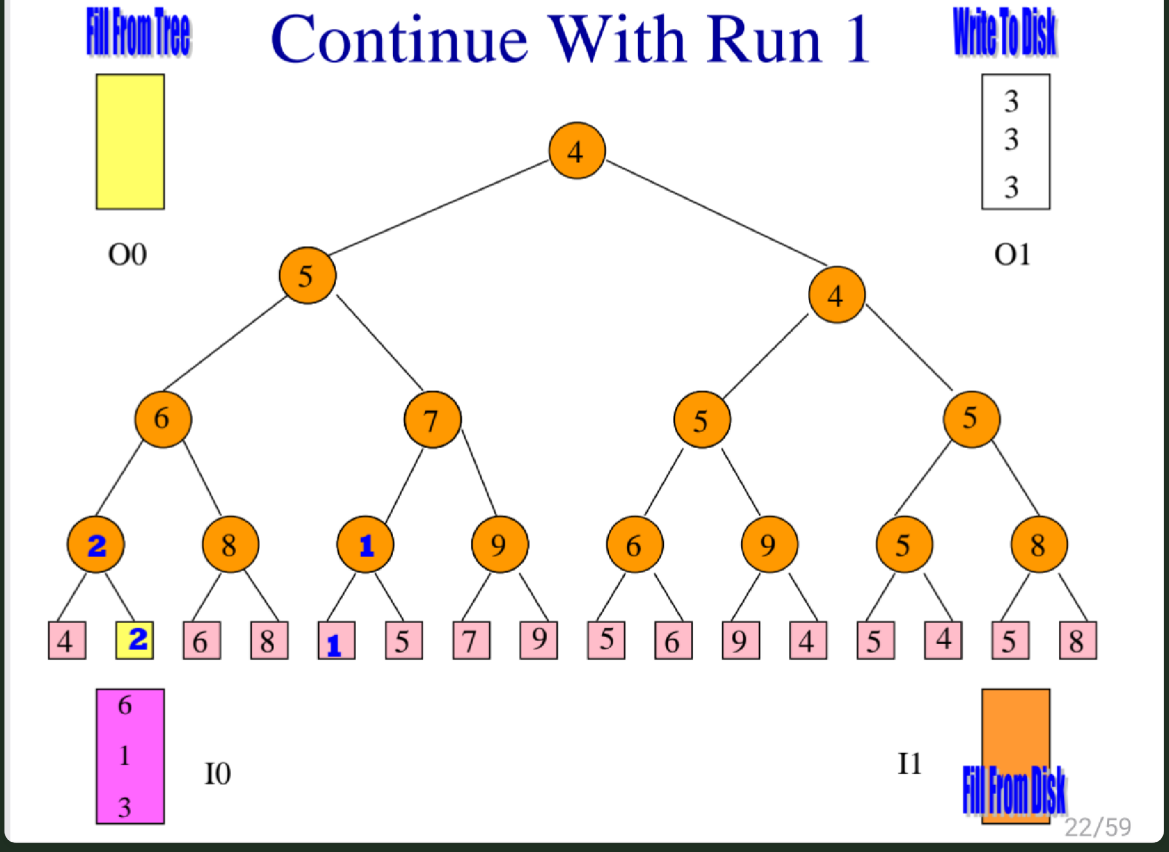

Write To Disk
Continue With Run 1
Fill From Tree
1
2
2
O0
3
O1
LHS winner.
RHS winner.
replay.
First step.
2nd step
remove minimum.
replace with next number
replay again.
Fill From Disk
I0
I1
1
9
2
freeze. carry this to run 2.
17/59

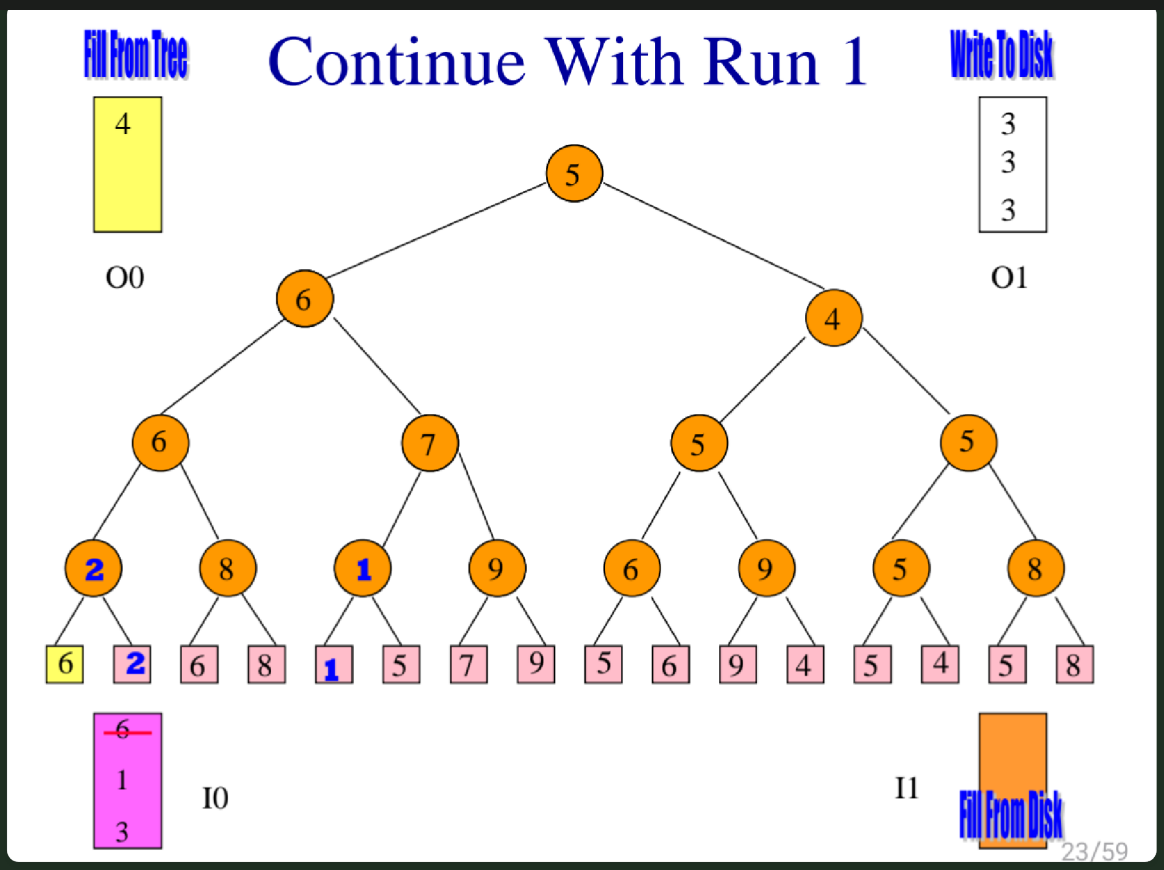

Write To Disk
Continue With Run 1
Fill From Tree
1
2
2
O0
3
3
O1
Fill From Disk
I0
I1
1
9
2
18/59

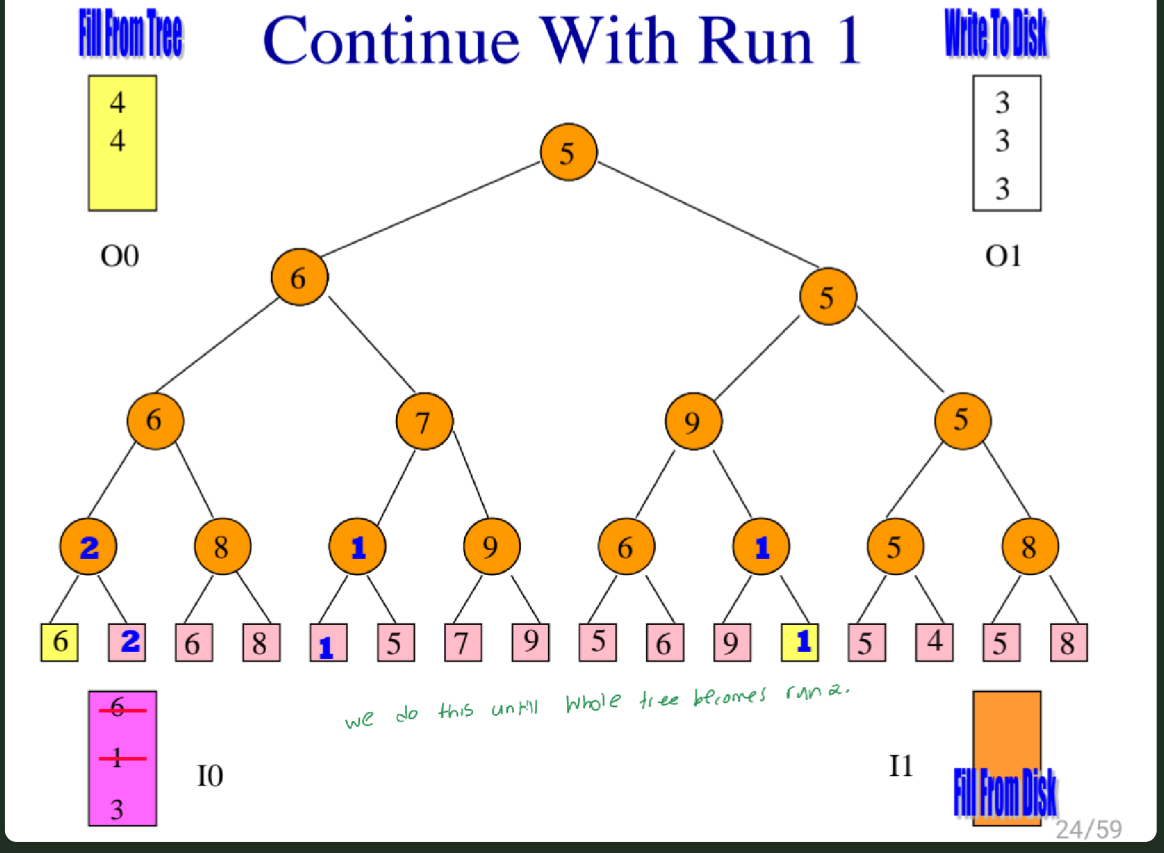

Write To Disk
Continue With Run 1
Fill From Tree
1
2
2
3
3
O0
O1
4
5
4
6
7
5
5
4
8
1
9
6
9
5
8
4
3
6
8
1
5
7
9
5
6
9
4
5
4
5
8
Fill From Disk
I0
I1
1
9
2


Write To Disk
Continue With Run 1
Fill From Tree
1
2
2
3
3
3
O0
O1
4
5
4
6
7
5
5
4
8
1
9
6
9
5
8
4
2
6
8
1
5
7
9
5
6
9
4
5
4
5
8
Fill From Disk
I0
Interchange Role Of Buffers
I1
1
9
2


Fill From Tree
Interchange Role Of Buffers
Write To Disk
3
3
3
O0
O1
4
5
4
6
7
5
5
4
8
1
9
6
9
5
8
4
2
6
8
1
5
7
9
5
6
9
4
5
4
5
8
6
1
3
I0
I1
Fill From Disk


Fill From Tree
Continue With Run 1
Write To Disk
3
3
3
O0
O1
4
5
4
6
7
5
5
2
8
1
9
6
9
5
8
4
2
6
8
1
5
7
9
5
6
9
4
5
4
5
8
6
1
3
I0
I1
Fill From Disk





Run-length and time analysis

- Let k be number of external nodes in loser tree.
- Run size >= k.
- Sorted input => 1 run.
- Reverse of sorted input => n/k runs.
- Average run size is ~2k.
- Total internal processing time using loser tree = O(n log k).

Input arrangement	Result
General case	Each completed run has at least k records, except possibly the final run
Sorted input	All records form one run (all records sorted nm 1 time ne, 2nd run one na)
Reverse-sorted input	Each run has about k records, producing about n/k runs (n -> number of records)
Random input	Average run length is approximately $2k$

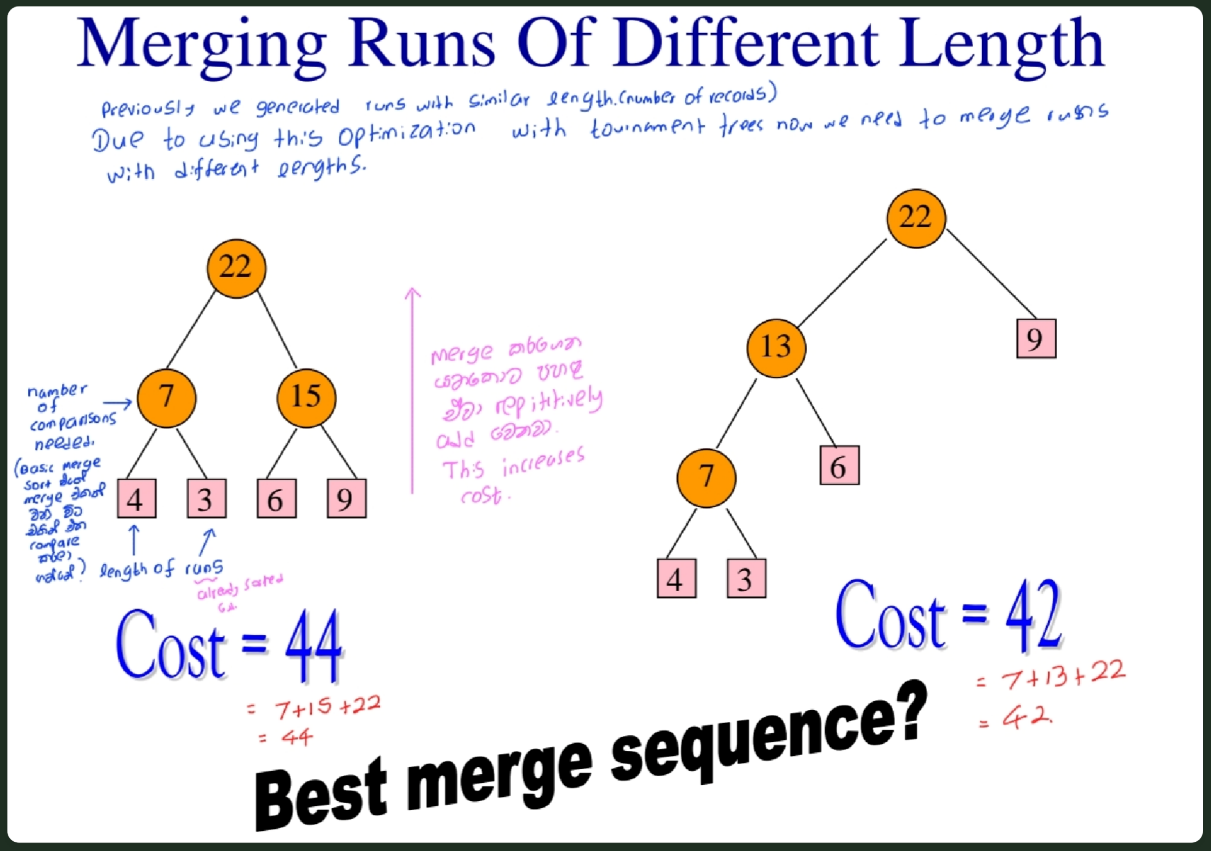
Every replacement replays one root-to-leaf path of the loser tree, taking $O(\log k)$ time. Across n records, total internal processing time is: (hama node ekakma buffer wlt danawa -> n time replays)

$$O(n \log k)$$

Why longer runs matter: doubling the average run length roughly halves the number of initial runs, which can reduce the number of later merge passes.

3. Optimal Merging and Weighted External Path Length

From here we are building the path to learn Huffman tree, (below are not tournament trees.)



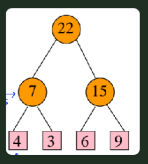
When runs have different lengths, merge order affects the total amount of work. *A short run placed near the top of a merge tree is processed repeatedly, while a run connected directly to the root is processed only once.*

> **Important**
>
> Let an external node represent an initial run, with its run length as the node's weight. The **weighted external path length (WEPL)** of a tree T is:
>
> $$\mathrm{WEPL}(T) = \sum_i w_i d_i$$
>
> where:
>
> - w_i is the weight or length of external run i.
> - d_i is the distance of that external node from the root.

WEPL equals the total merge cost because a record is read and merged once for every merge level on the path from its initial run to the root.

Example: runs of lengths $4, 3, 6,$ and 9

A balanced merge order gives:

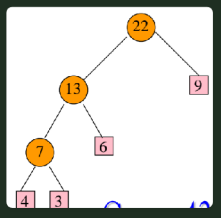



```text
4 + 3 = 7
6 + 9 = 15
7 + 15 = 22
Total merge cost = 7 + 15 + 22 = 44
```

Equivalently:

$$\text{WEPL} = 4(2) + 3(2) + 6(2) + 9(2) = 44$$

A better merge order combines the smallest runs first:

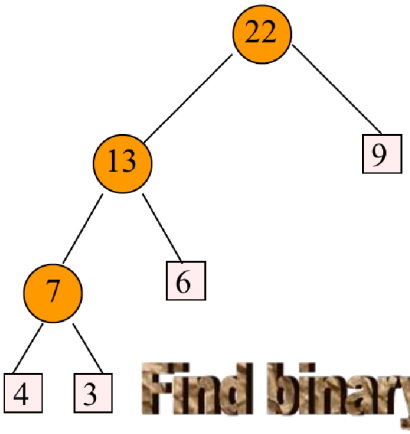



```text
3 + 4 = 7
6 + 7 = 13
9 + 13 = 22
Total merge cost = 7 + 13 + 22 = 42
```

$$\text{WEPL} = 4(3) + 3(3) + 6(2) + 9(1) = 42$$

Weighted External Path Length

WEPL(T) = Σ(weight of external node i)
* (distance of node i from root of T)



WEPL(T) = 4 * 3 + 3*3 + 6*2 + 9*1
= 42
= Merge Cost

Find binary tree with minimum WEPL.

The second tree is better because the longest run, of length 9, is merged only once.

> Tip
> The problem of finding a binary merge tree with minimum WEPL is exactly the problem solved by a Huffman tree.

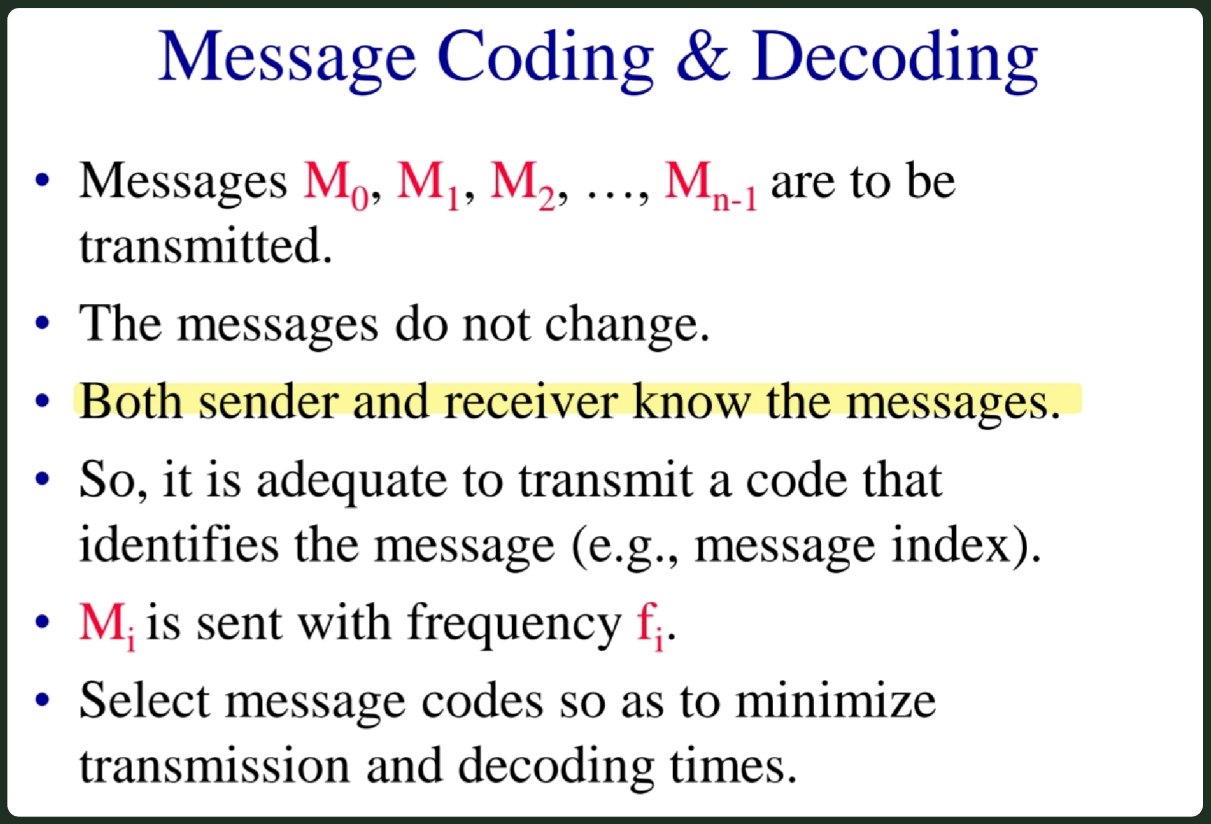
4. Other applications of optimal merge trees

Other Applications

- Message coding and decoding.
- Lossless data compression.

Optimal merge trees also solve message-coding and lossless-compression problems.

Message coding and decoding



If message M_i has a code of length l_i, its contribution to the transmission cost is $f_i l_i$. Therefore:

$$\text{transmission cost} = \sum_i f_i l_i$$

A binary decoding tree assigns `0` to one branch and `1` to the other. Each message is stored at an external node, so its code length equals its depth. Consequently:

$$\text{transmission cost} = \text{decoding cost} = \text{WEPL}$$



Prefix property

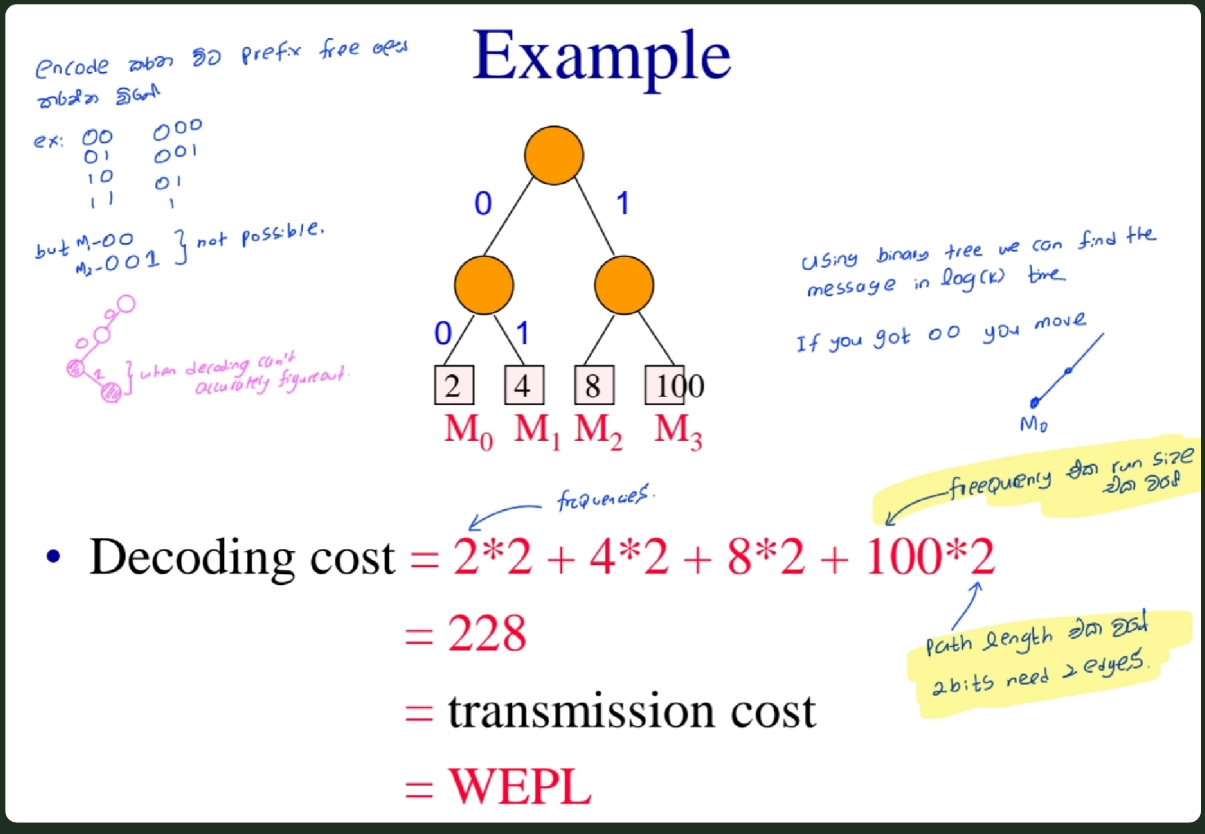
A valid variable-length code must be **prefix-free**: no complete code may be the prefix of another code. Otherwise, the decoder would not know where one code ends.

Every binary tree whose messages occupy only leaves defines a prefix-free code. Starting at the root, follow the edges labelled `0` and `1` until reaching a message leaf.
-> binary tree approach doesn't have this prefix issue, things leaf nodes like 00 and 001 both can't be exist together

Message-frequency example

- $n = 4$ messages.
- The frequencies are [2, 4, 8, 100].
- Use 2-bit codes [00, 01, 10, 11].
- Transmission cost $= 2*2 + 4*2 + 8*2 + 100*2$
  $= 228$.
- Decoding is done using a binary tree.

$$2(2) + 4(2) + 8(2) + 100(2) = 228$$



A variable-length prefix code can use:

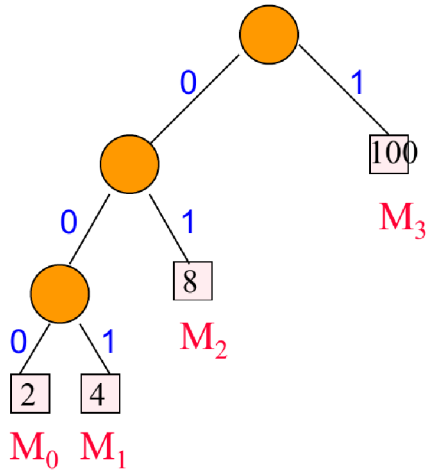
Message	Message frequency	Code	Code length
M_1	2	`000`	3
M_2	4	`001`	3
M_3	8	`01`	2
M_4	100	`1`	1

Its cost is:

$$2(3) + 4(3) + 8(2) + 100(1) = 134$$

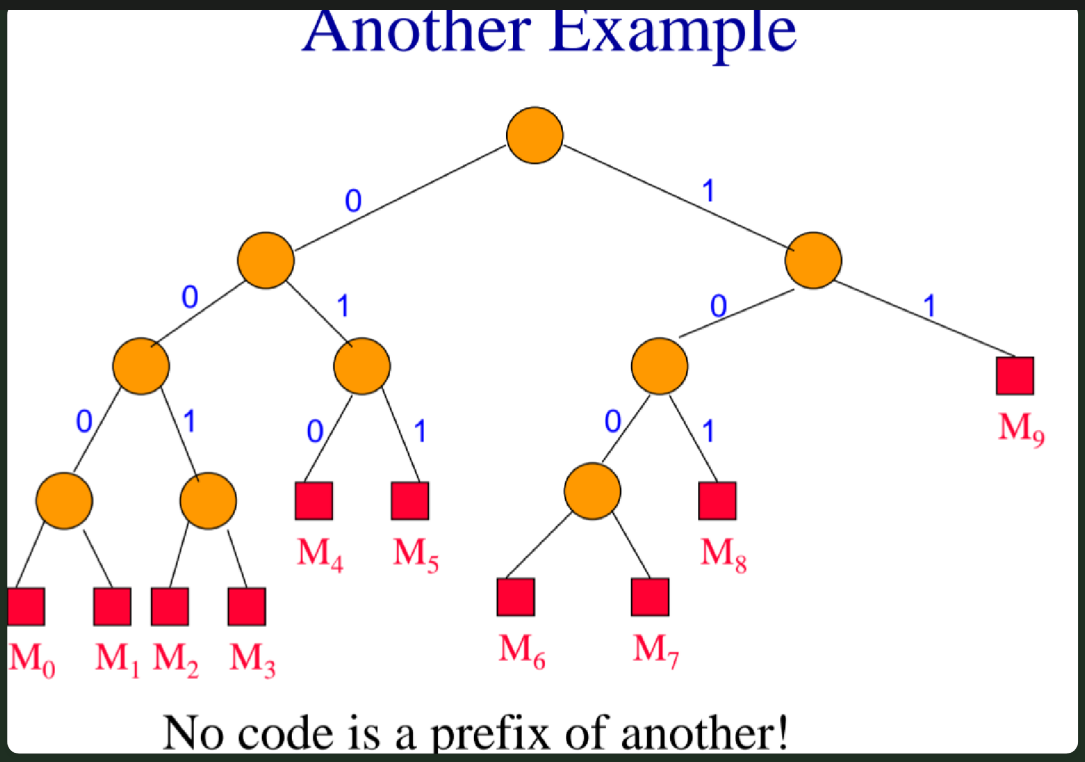
Example

- Every binary tree with n external nodes defines a code set for n messages.



- Decoding cost
  = 2*3 + 4*3 + 8*2 + 100*1
  = 134
  = transmission cost
  = WEPL

The frequent message receives the shortest code, which reduces the weighted cost from 228 to 134. -> *so using minimized WEPL reduces transmission cost as well as decoding cost*



Lossless-compression

Suppose an alphabet has the following symbol counts:

Symbol	Frequency
a	10
b	5
c	100
d	900

Using a fixed two-bit code requires:

$$10(2) + 5(2) + 100(2) + 900(2) = 2030 \text{ bits}$$

Using the prefix code `a = 000`, `b = 001`, `c = 01`, and `d = 1` requires:

$$10(3) + 5(3) + 100(2) + 900(1) = 1145 \text{ bits}$$

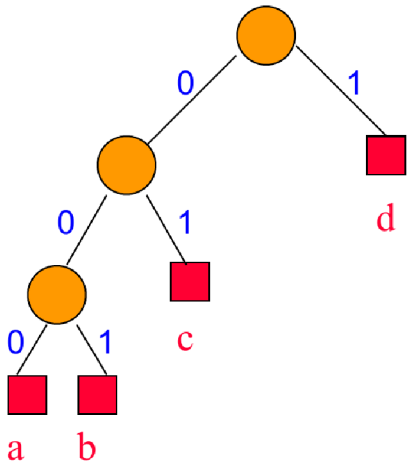
The compression ratio, excluding the code table, is approximately:

$$\frac{2030}{1145} \approx 1.8$$

For example, the bit string `0001100101...` is decoded as:

```text
000 | 1 | 1 | 001 | 01 | ...
 a  | d | d |  b  | c  | ...
```

Lossless Data Compression



- Decode 0001100101…
- addbc…
- Compression ratio is maximized when the decode tree has minimum WEPL.

Maximum compression is obtained when the decoding tree has minimum WEPL.

5. Huffman Trees

Huffman Trees

- Trees that have minimum WEPL.
- Binary trees with minimum WEPL may be constructed using a greedy algorithm.
- For higher order trees with minimum WEPL, a preprocessing step followed by the greedy algorithm may be used.
- Huffman codes: codes defined by minimum WEPL trees.

using a greedy algorithm

Caution

Huffman Tree is not a tournament tree

	Huffman Tree	Tournament Tree
Purpose	Data compression / encoding	Finding min/max efficiently
Leaves	Symbols with frequencies	Elements being compared
Internal nodes	Combined frequencies	Winner of comparisons
Construction	Repeatedly combine the two smallest frequencies	Pairwise comparisons between elements
Main use	Huffman coding	Selection, sorting, k-way merging

1. Binary Huffman Trees

A **Huffman tree** is a tree with minimum weighted external path length. Its leaf depths define **Huffman codes**.

> **Note**
> For a binary tree, a greedy algorithm constructs an optimal tree:
>
> 1. Start with one single-node tree for every weight.
> 2. Remove the two trees with the smallest weights.
> 3. Create a new root with those trees as its children. -> number of total trees will be reduced by 1 in each tree creation.
> 4. Give the new tree a weight equal to the sum of its children's weights.
> 5. Insert the new tree back into the collection.
> 6. Repeat until only one tree remains.

Worked example: weights `[2, 5, 4, 7, 9]`

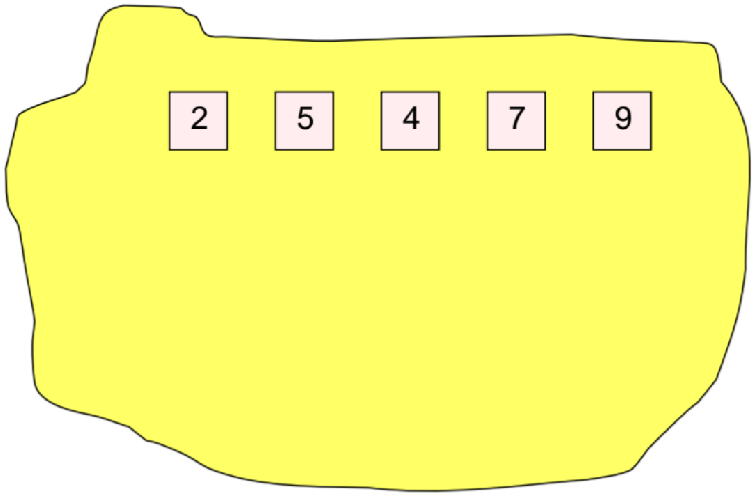
```text
2 + 4  = 6
5 + 6  = 11
7 + 9  = 16
11 + 16 = 27
```

The WEPL is the sum of the combined weights:

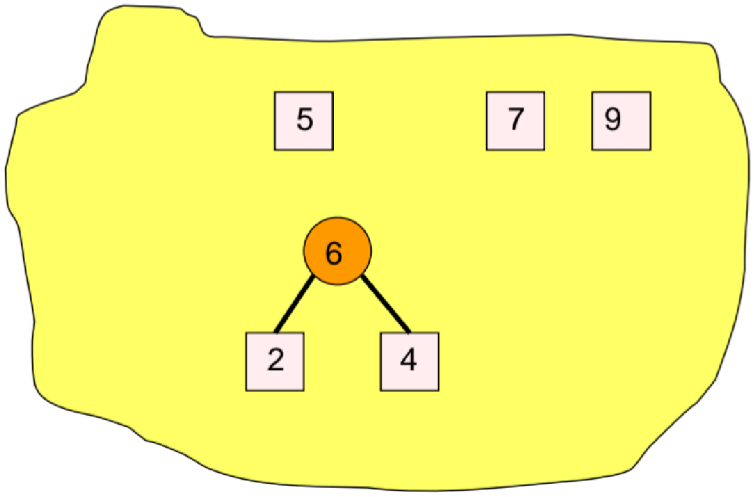
$$6 + 11 + 16 + 27 = 60$$

Example

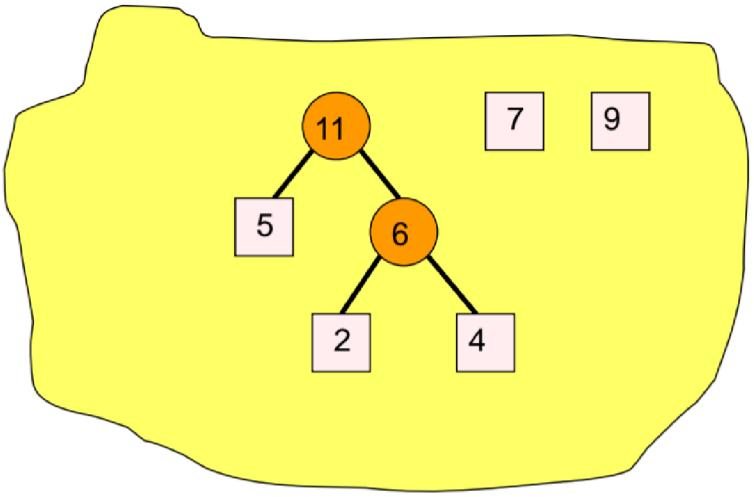
- n = 5, w[0:4] = [2, 5, 4, 7, 9].



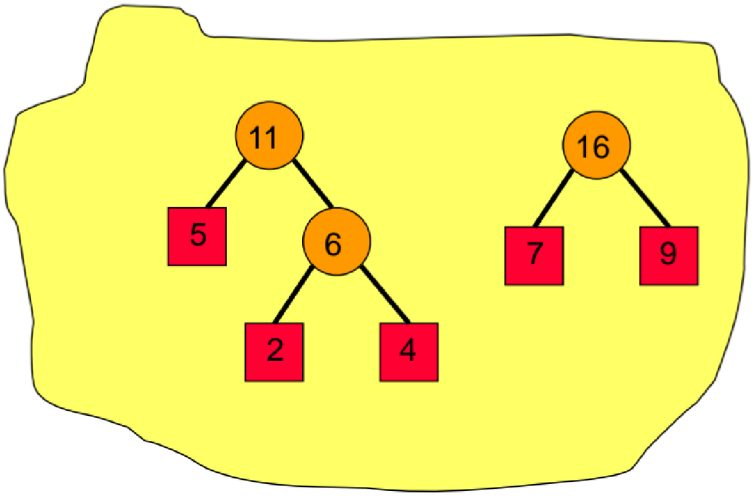
- n = 5, w[0:4] = [2, 5, 4, 7, 9].



- n = 5, w[0:4] = [2, 5, 4, 7, 9].

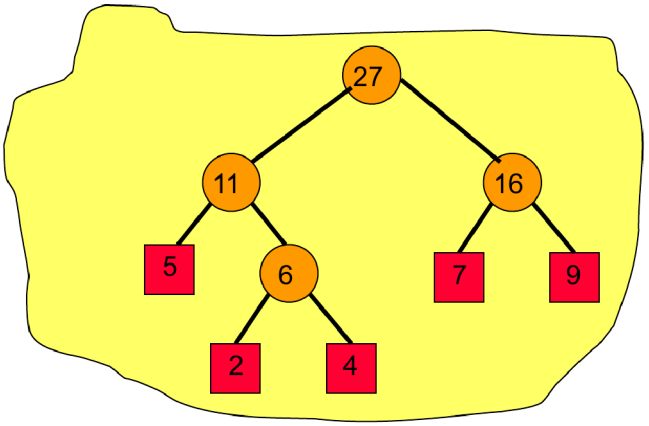


- n = 5, w[0:4] = [2, 5, 4, 7, 9].



Example

- n = 5, w[0:4] = [2, 5, 4, 7, 9].



Data structure and complexity

A min heap supports the required tree-collection operations: weights tika min heap ekaka dala minimum deka hoynw

- Operations are:
  - Initialize with n trees.
  - Remove 2 trees with least weight.
  - Insert new tree.
- Use a min heap.
- Initialize … O(n).
- 2(n – 1) remove min operations … O(n log n).
- n – 1 insert operations … O(n log n).
- Total time is O(n log n).
- Or, (n – 1) remove mins and (n – 1) change mins.

Operation	Number of operations	Cost
Initialize the heap	1	$O(n)$
Remove minimum	$2(n-1)$	$O(n \log n)$ total
Insert combined tree	$n-1$	$O(n \log n)$ total

Heap Data Structure

In a min heap, the smallest tree is stored at the root.

Reading the minimum is $O(1)$, but removing it requires restoring the heap property:

1. Remove the root.
2. Move the last heap element to the root.
3. Compare it with its children.
4. Swap it with the smaller child when necessary.
5. Continue downward until the heap property is restored.

This process is called **sift-down** or **heapify-down**.

A binary heap containing n elements is a complete binary tree with height:

$$\lfloor \log_2 n \rfloor$$

The replacement element may travel from the root to a leaf, performing constant work at each level. Therefore:

$$O(\text{height}) = O(\log n)$$

So:

Operation	Complexity
Read the minimum	$O(1)$
Remove the minimum	$O(\log n)$

For n initial weights:

1. Build the min heap: $O(n)$
2. Repeat until only one tree remains:
   - Remove the smallest tree: $O(\log n)$
   - Remove the second-smallest tree: $O(\log n)$
   - Combine them.
   - Insert the combined tree: $O(\log n)$

Each merge reduces the number of trees by one. Starting with n trees and finishing with one requires:

$$n - 1 \text{ merges}$$

Each merge performs three heap operations:

$$2 \text{ remove-min} + 1 \text{ insertion}$$

Therefore:

$$\begin{aligned} T(n) &= O(n) + (n-1)\left[2O(\log n) + O(\log n)\right] \\ &= O(n) + 3(n-1)O(\log n) \\ &= O(n \log n) \end{aligned}$$

Therefore, binary Huffman-tree construction takes:

$$O(n \log n)$$

2. Higher-Order Huffman Trees

Huffman tree only gives what's the best possible way to select runs to do a k-way merge in our merge sort context. -> improve merging (stage 2) by reducing cost.

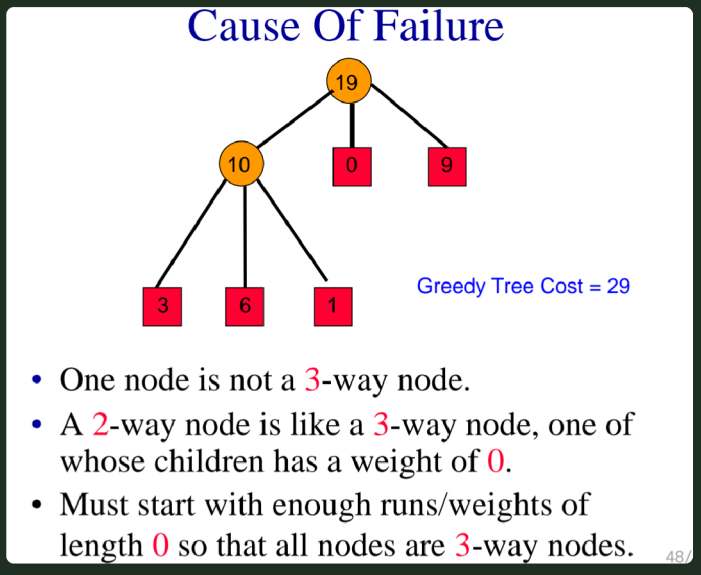
Tournament tree -> can be used to generate longer runs (improvement) and to do k-way merge

For a k-ary tree with $k > 2$, repeatedly combining the k smallest weights works only when the final tree can be a **full k-ary tree**, meaning every internal node has exactly k children.

Why the naïve greedy method can fail

For a three-way tree with weights `[3, 6, 1, 9]`, immediately combining the three smallest real weights gives:

```text
1 + 3 + 6 = 10
10 + 9 = 19
Cost = 10 + 19 = 29
```



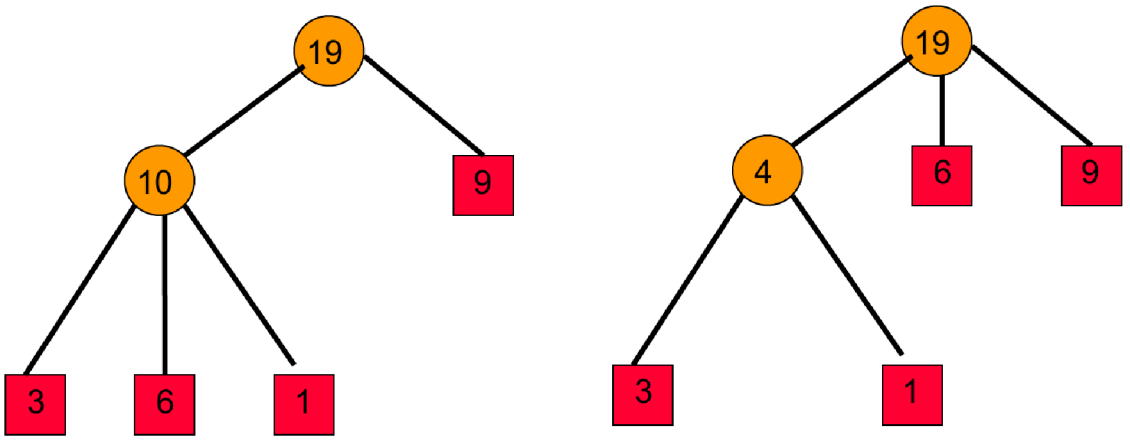
The final root has only two children. Treating the missing child as a zero-weight dummy gives a better construction:

```text
```

```
0 + 1 + 3 = 4
4 + 6 + 9 = 19
Cost = 4 + 19 = 23
```

Higher Order Trees

- Greedy scheme doesn't work!
- 3-way tree with weights [3, 6, 1, 9].



Greedy Tree Cost = 29

Optimal Tree Cost = 23

Number of zero-weight dummy nodes

Let:

- r be the initial number of real weights or runs.
- q be the number of zero-weight dummy runs added.
- k be the tree order.
- s be the number of k-way merges.

Each K-way merge reduces the number of trees by $k-1$ (by removing K trees and adding 1 tree). To finish with exactly one tree:

$$total\ trees = r + q - s(k-1) = 1$$

Therefore:

- So, we want
  $r + q - s(k-1) = 1$
  for some positive integer s.
- So, $r + q - 1 = s(k - 1)$.
- Or, $(r + q - 1) \bmod (k - 1) = 0$.
- Or, $r + q - 1$ is divisible by $k - 1$.
  - This implies that $q < k - 1$.
- $(r - 1) \bmod (k - 1) = 0 \Rightarrow q = 0$.
- $(r - 1) \bmod (k - 1) \neq 0 \Rightarrow$
  $q = (1 - r) \bmod (k - 1)$.

$$(r + q - 1) \bmod (k - 1) = 0$$

The least non-negative valid value is:

$$q = (1 - r) \bmod (k - 1), \qquad 0 \leq q < k - 1$$

Examples:

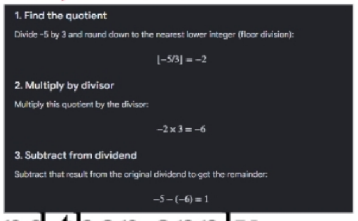
- For $k = 2$, $k - 1 = 1$, so $q = 0$ for every r. Binary Huffman coding never needs dummy weights.
- For $k = 4$ and $r = 6$:

$$q = (1 - 6) \bmod 3 = (-5) \bmod 3 = 1$$

One dummy run is added, giving seven starting runs before applying the four-way greedy algorithm.

Examples

- $k = 2$. two way merge → we don't need to add any 0 length runs.
  - $q = (1 - r) \bmod (k - 1) = (1 - r) \bmod 1 = 0$.
  - So, no runs of length 0 are to be added.
- $k = 4, r = 6$.
  - $q = (1 - r) \bmod (k - 1) = (1 - 6) \bmod 3$
    $= (-5) \bmod 3$
    $= (6 - 5) \bmod 3$
    $= 1$.
  - So, must start with 7 runs, and then apply greedy method.



3. Lab

Lab 02: Huffman Coding — Encode and Decode

Duration: 2 Hours | Language: C++

1. Learning Objectives

- Build a Huffman tree from character frequencies using a min-heap (greedy algorithm).
- Generate prefix-free binary codes by traversing the Huffman tree.
- Encode a text message into a binary string using the generated codes.
- Decode a binary string back to the original text by traversing the Huffman tree.

2. Background

What is Huffman Coding?
Huffman coding is a lossless data compression algorithm. It assigns variable-length binary codes to characters based on their frequencies: frequent characters get shorter codes, rare characters get longer codes. This minimizes the total number of bits needed to represent a message.

The Greedy Algorithm:
Start with each character as a leaf node with its frequency. Use a min-heap to repeatedly extract the two nodes with the smallest frequencies, combine them into a new internal node whose frequency is their sum, and push the new node back. When one node remains, it is the root of the Huffman tree.

Code Generation:
Traverse the tree from root to each leaf. Each left edge = "0", each right edge = "1". The path from root to a leaf gives that character's binary code. Because no leaf is an ancestor of another, the codes are *prefix-free*.

WEPL (Weighted External Path Length):
WEPL = ∑(frequency × code length) over all characters. Huffman's algorithm produces the minimum possible WEPL.

Example:
Frequencies: a=5, b=2, r=2, c=1, d=1
Step 1: Combine c(1) and d(1) → internal node (2).
Step 2: Combine two smallest again, and continue until one root remains.
Result: codes like a=0, b=111, c=100, d=101, r=110 → WEPL = 23 bits.

---

3. Your Task

Implement a Huffman coding system by completing the **HuffmanTree** class. You need to implement:

Method	What to implement	Time Guide
`CompareNode`	Min-heap comparator: smaller frequency = higher priority	5 min
`build(frequencies)`	Build the Huffman tree using a min-heap (greedy algorithm)	30 min
`generateCodes(node, code)`	Recursively traverse tree: left="0", right="1", store code at each leaf	15 min
`encode(text)`	Look up each character's code and concatenate into a binary string	10 min
`decode(bits)`	Traverse tree bit by bit, output character at each leaf, reset to root	20 min

4. Requirements

- Codes must be **prefix-free** (no code is a prefix of another).
- WEPL must be **optimal** (minimum possible).
- `decode(encode(message))` must return the **original message**.
- Handle edge cases: **single character** (assign code "0"), **two characters** (codes "0" and "1").

---

5. Hints

1. Draw the Huffman tree for a=5, b=2, r=2, c=1, d=1 on paper first. Trace the build process step by step.
2. In C++, `priority_queue` is a **max-heap** by default. Your `CompareNode` must return `true` when the first argument should have *lower* priority (higher frequency) so it acts as a min-heap.
3. For the single-character case, the root is a leaf with no children. Assign it code "0" so encoding and decoding still work.
4. The `decode` function must reset to the root **every time** it reaches a leaf, not after a fixed number of bits.
5. Use `priority_queue<HuffmanNode*, vector<HuffmanNode*>, CompareNode>` for the min-heap.

---

6. Important Notes

- Your `encode()` returns a `string` of '0's and '1's.
- Your `decode()` returns the original text `string`.
- You do **NOT** need to print or output anything — the test harness calls your functions and checks the results.
- Do **NOT** include `main()`, `#include` headers, or `using namespace std;` — these are provided by the test harness.
- Since Huffman trees are not unique (tie-breaking can vary), the tests check *properties* (prefix-free, optimal WEPL, correct decode) rather than exact code strings.

c++

```cpp
//
============================================================
======
// Struct: HuffmanNode
// Represents a node in the Huffman tree.
//
============================================================
======
struct HuffmanNode {
```

```
    char ch;           // character ('\0' for internal
nodes)
    int freq;          // frequency
    HuffmanNode* left;
    HuffmanNode* right;

    // Leaf node constructor
    HuffmanNode(char c, int f) : ch(c), freq(f),
left(nullptr), right(nullptr) {}

    // Internal node constructor
    HuffmanNode(int f, HuffmanNode* l, HuffmanNode* r)
        : ch('\0'), freq(f), left(l), right(r) {}

    bool isLeaf() const { return !left && !right; }
};

// Min-heap comparator: smaller frequency has higher
priority.
// TODO: Complete the comparator for the priority
queue.
//       Return true if a should have LOWER priority
than b.
//       (priority_queue is a max-heap by default, so
returning
//        a->freq > b->freq makes it a min-heap.)
struct CompareNode {
    bool operator()(const HuffmanNode* a, const
HuffmanNode* b) const {
        return a->freq > b->freq;
    }
};

//
==========================================================
======
// Class: HuffmanTree
// Builds a Huffman tree, generates codes, encodes and
decodes.
```

```
// ============================================================
class HuffmanTree {
private:
    HuffmanNode* root;
    map<char, string> codes;

    // Recursively generate codes by traversing the tree.
    // Left edge = "0", Right edge = "1".
    void generateCodes(HuffmanNode* node, const string& code) {
        // TODO: 1. Base case: if node is null, return.
        //       2. If node is a leaf:
        //          - Store the code: codes[node->ch] = code
        //          - Special case: if code is empty (single char tree),
        //            store "0" instead.
        //          - Return.
        //       3. Recurse left with code + "0".
        //       4. Recurse right with code + "1".
        if (!node) {
            return;
        }

        if (node->isLeaf()) {
            if (code.empty()){
                codes[node->ch] ="0";
            }
            else{
                codes[node->ch] = code;
            }
            return;
        }

        generateCodes(node->left, code + "0");
```

```
        generateCodes(node->right, code + "1");
    }

    // Free all nodes in the tree (destructor helper).
    void freeTree(HuffmanNode* node) {
        if (!node) return;
        freeTree(node->left);
        freeTree(node->right);
        delete node;
    }

public:
    HuffmanTree() : root(nullptr) {}

    ~HuffmanTree() {
        freeTree(root);
    }

    // Build Huffman tree from character-frequency
pairs using a min-heap.
    void build(const vector<pair<char, int>>&
frequencies) {
        // TODO: 1. Create a min-heap (priority_queue
with CompareNode).
        //       2. Push a new HuffmanNode(char, freq)
for each pair.
        //       3. While the heap has more than one
node:
        //          a. Pop the two nodes with smallest
frequency.
        //          b. Create a new internal node
with:
        //             - freq = sum of the two
children's frequencies
        //             - left = first popped node
        //             - right = second popped node
        //          c. Push the new node back onto the
heap.
```

```
        //       4. The last remaining node is the
root.
        //       5. Call generateCodes(root, "") to
build the code table.
        freeTree(root);
        root = nullptr;
        codes.clear();

        priority_queue<
            HuffmanNode*,
            vector<HuffmanNode*>,
            CompareNode
        > pq;

        for (const auto& item : frequencies) {
            if (item.second > 0) {
                pq.push(new HuffmanNode(item.first,
item.second));
            }
        }

        if (pq.empty()) {
            return;
        }

        while (pq.size() > 1) {
            HuffmanNode* first = pq.top();
            pq.pop();

            HuffmanNode* second = pq.top();
            pq.pop();

            HuffmanNode* parent = new HuffmanNode(
                first->freq + second->freq,
                first,
                second
            );

            pq.push(parent);
```

```
        }

        root = pq.top();
        generateCodes(root, "");
    }

    // Return the generated prefix-free codes.
    map<char, string> getCodes() const {
        return codes;
    }

    // Encode a text string into a binary string using
the Huffman codes.
    string encode(const string& text) const {
        // TODO: For each character in text, look up
its code in the
        //       codes map and append it to the result
string.
       string result;

        for (char c : text) {
            auto it = codes.find(c);
            result += it->second;
        }

        return result;
    }

    // Decode a binary string back to the original
text using the Huffman tree.
    string decode(const string& bits) const {
        // TODO: 1. Start at the root.
        //       2. Special case: if root is a leaf
(single char),
        //          return that char repeated
bits.size() times.
        //       3. For each bit in the binary string:
        //          - '0' -> go left
        //          - '1' -> go right
```

```
        //          - If you reach a leaf, append its
character to
        //            the result and reset to the
root.
        //       4. Return the decoded string.
        if (!root) {
            return "";
        }

        if (root->isLeaf()) {
            return string(bits.size(), root->ch);
        }

        string result;
        HuffmanNode* current = root;

        for (char bit : bits) {
            if (bit == '0') {
                current = current->left;
            } else if (bit == '1') {
                current = current->right;
            }


            if (current->isLeaf()) {
                result += current->ch;
                current = root;
            }
        }


        return result;
    }
};
```

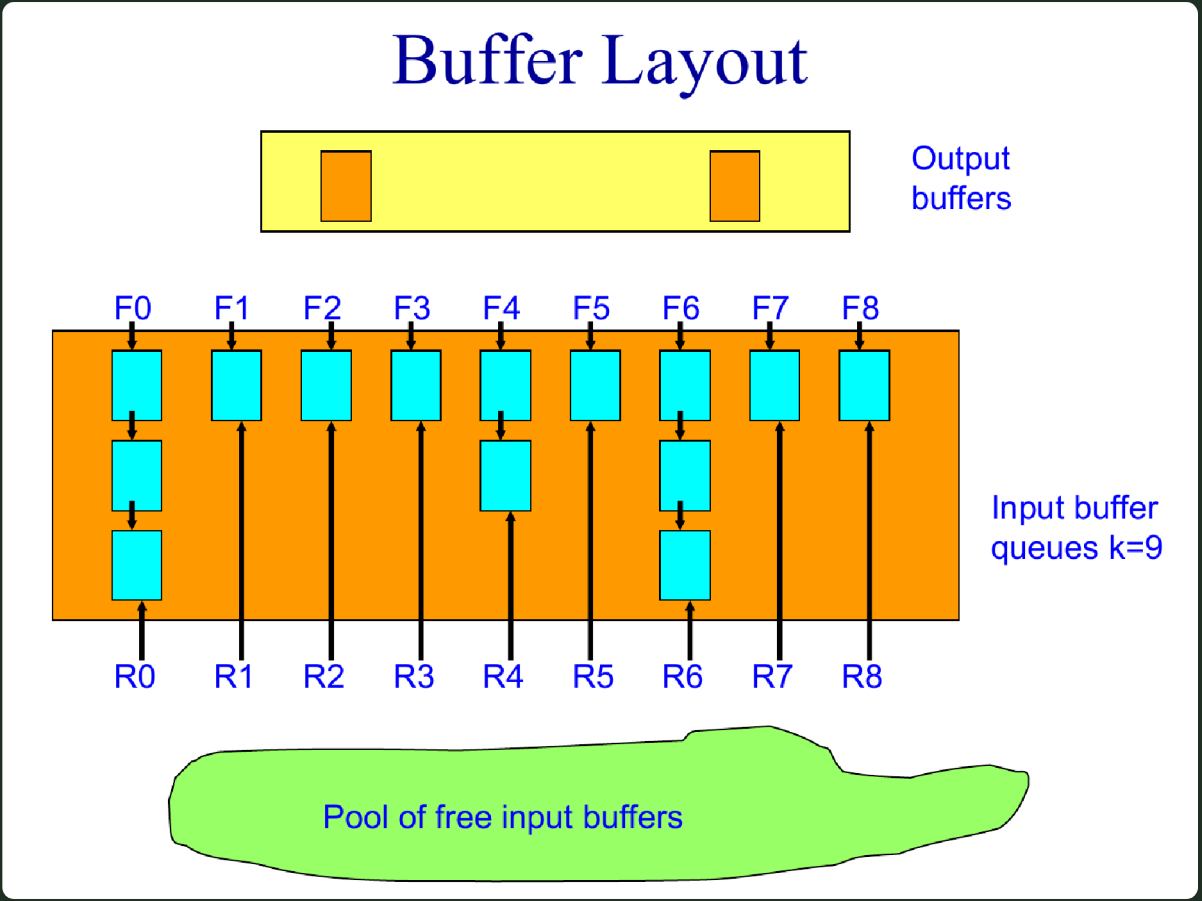
6. Buffering an External k-Way Merge

An efficient k-way merge must keep the CPU busy while disk reads and writes occur in the background.

Memory partition

- Exactly two output buffers are used so one can be written while the other is filled.
- At least $k+1$ input buffers are required for a k-way merge. as input
- A total of $2k$ input buffers is sufficient when buffers are allocated dynamically.
- The remaining memory is used by the loser tree and control structures.

Permanently assigning two input buffers to every run is ineffective: one run may need several prefetched blocks while buffers assigned to other runs remain unused. Instead, each run has a queue of available buffers and all unassigned buffers are kept in a shared free pool.



Predicting which run will exhaust first

To decide where the next free input buffer should be allocated:

1. Examine the key of the last record currently buffered from each run.
2. The run with the smallest last key is expected to exhaust its available buffered data first.
3. Use a consistent tie-breaker if multiple runs have the same last key.
4. Prefetch the next block of that run into a free buffer.

Because every run is sorted, this rule identifies the run whose buffered range will be consumed earliest by the merge.

Initialization with $2k$ input buffers

1. Create k input-buffer queues, one for each run.
2. Read one block from each run and place it in that run's queue. This uses k buffers.
3. Put $k-1$ unused buffers into the shared free pool.
4. Use the remaining buffer to begin an asynchronous read from the first run predicted to exhaust.
5. Set `activeOutputBuffer = 0`.
6. Initialize the min loser tree with the first record from every run.

The `kWayMerge` step

The CPU repeatedly selects the loser-tree winner and copies it to the active output buffer. This step stops when:

- The output buffer becomes full, or
- An end-of-run key is merged into the output buffer (very large value to indicate end of run).

If an input buffer becomes empty during the merge, advance to the next buffer in that run's queue and return the empty buffer to the free pool.

```text
repeat
    kWayMerge into active output buffer
    wait for outstanding input/output to complete

    if an input block completed:
        add its buffer to the queue for its run
```

```
    determine which run will exhaust first
    if that run has more input:
        start reading its next block into a free
buffer

    start writing the active output buffer
    activeOutputBuffer = 1 - activeOutputBuffer
until an end-of-run sentinel is merged
```

The loser tree selects each next record in $O(\log k)$ time, so merging N records requires $O(N \log k)$ internal processing. Buffering does not change this asymptotic CPU bound; it reduces elapsed time by overlapping CPU work with disk I/O.

> ⚠ Caution
> The **"Buffering an External k-Way Merge"** section does not use a Huffman tree to select individual records.
> It uses a **min loser tree**:
>
> - Each external node represents the current record from one run.
> - The winner is the smallest available record.
> - After outputting it, the record is replaced with the next record from the same run.
> - Replay takes $O(\log k)$.
>
> The Huffman tree and loser tree solve different problems:

Structure	Purpose
Huffman tree	Determines the optimal order in which runs of different lengths should be merged
Min loser tree	Performs one selected k-way merge by repeatedly finding the smallest record
Input/output buffers	Overlap disk access with the loser-tree merge

For example, a Huffman tree might determine:

```
First merge:  R1, R2, R3
Second merge: result, R4, R5
```

During each of those merge operations, a loser tree and the buffering strategy perform the actual record-by-record merging.
So the relationship is:

```
Huffman tree
    ↓ decides the merge schedule

Loser tree + buffers
    ↓ execute each merge efficiently

Final sorted run
```

The buffering section can operate without Huffman coding—it assumes the k runs to merge have already been selected.



7. Quick Revision

Complexity and formula summary

Concept	Result
Average replacement-selection run length	Approximately $2k$
Worst input order for replacement selection	Reverse sorted, about n/k runs
Replacement-selection CPU time	$O(n \log k)$
Weighted external path length	$\sum_i w_i d_i$
Binary Huffman construction	$O(n \log n)$ using a min heap
Dummy nodes for a full k-ary tree	$q = (1 - r) \bmod (k - 1)$
Minimum input buffers for a k-way merge	$k + 1$
Sufficient dynamically allocated input buffers	$2k$
Output buffers	Exactly 2
Loser-tree k-way merge CPU time	$O(N \log k)$

Key takeaways

1. Replacement selection uses a loser tree to generate runs longer than the available in-memory tree capacity.
2. Frozen records wait for the next run whenever they would violate the order of the current run.
3. For unequal run lengths, total merge cost equals the merge tree's WEPL.
4. Huffman's greedy algorithm constructs a minimum-WEPL binary tree by repeatedly combining the two smallest weights.
5. Prefix-free Huffman codes give frequent messages short codes and minimize weighted transmission and decoding cost.
6. Higher-order Huffman trees may require zero-weight dummy nodes before applying the greedy algorithm.
7. A shared input-buffer pool is more effective than permanently assigning the same number of buffers to every run.

8. References

- E. Horowitz, S. Sahni, and D. Mehta, *Fundamentals of Data Structures in C++*, 2nd ed., Silicon Press, 2007.
- Lecture slides of Prof. Sartaj Sahni.